

# SQLP 과목 III 튜닝 스타일 모의문제 · 8회차

SQL 자격검정 실전문제 3장 스타일 · 4지선다 · 총 20문항 · 원본 창작

1

데이터베이스의 저장 구조(세그먼트·익스텐트·블록)와, 갱신으로 행이 커질 때 나타나는 행 이전(Row Migration)·행 연쇄(Row Chaining)에 대한 설명으로 가장 적절하지 않은 것은?

- ① 테이블·인덱스처럼 저장 공간을 차지하는 오브젝트 하나는 원칙적으로 하나의 세그먼트에 대응하고, 그 세그먼트가 쓰는 공간은 여러 익스텐트로 나뉘어 익스텐트끼리는 물리적으로 서로 떨어진 자리에 놓일 수도 있다.
- ② 행 연쇄(Row Chaining)는 한 행이 블록 하나에 다 담기지 못할 만큼 커서 처음 저장될 때부터 둘 이상의 블록에 나뉘어 저장되는 현상이며, 이런 행을 읽으려면 연쇄된 블록을 모두 방문해야 해 블록 I/O가 늘어난다.
- ③ 행 이전(Row Migration)은 갱신으로 행이 커져 원래 블록의 남은 공간에 다시 들어가지 못할 때, 행을 다른 블록으로 옮기고 원래 자리에는 옮겨 간 위치를 가리키는 포인터만 남겨 두는 현상이다.
- ④ 행 이전이 일어나면 그 행을 가리키던 인덱스의 ROWID가 새로 옮겨 간 블록을 가리키도록 함께 갱신되므로, 인덱스로 그 행을 찾을 때 원래 블록을 거치지 않고 곧바로 옮겨 간 블록에 도달한다.

2

블록을 디스크에서 읽어 오기까지 기다리는 I/O 대기 이벤트(db file sequential read·db file scattered read)와, 이미 처리 중인 블록을 놓고 세션들이 부딪혀 생기는 경합성 대기 이벤트(buffer busy waits·read by other session)에 대한 설명으로 가장 적절한 것은?

- ① db file sequential read가 길게 잡히면 그 값의 대부분은 여러 세션이 같은 블록을 놓고 다투는 경합에서 비롯되므로, 대기 이벤트 이름이 가리키는 디스크 읽기보다 블록 경합을 먼저 손봐야 사라진다.
- ② buffer busy waits는 원하는 블록이 버퍼 캐시에 없어 디스크에서 읽어 들이는 것을 기다리는 대기이므로, 버퍼 캐시를 키워 적중률을 높이면 이 대기의 근본 원인이 없어진다.
- ③ read by other session은 내가 필요로 하는 블록을 마침 다른 세션이 디스크에서 캐시로 올리는 중이어서, 같은 블록을 중복해 읽지 않고 그 읽기가 끝나기를 기다릴 때 발생하며, 같은 블록에 요청이 몰릴 때 나타나는 경합성 대기다.
- ④ buffer busy waits와 read by other session은 둘 다 디스크 장비의 응답 지연을 그대로 나타내는 순수 I/O 대기이므로, 대기 시간이 길면 스토리지 성능부터 점검하는 것이 정석이다.

모바일 뱅킹 푸시 알림 발송 배치가 발송 대상자 80만 명을 순회하며 대상자 1명당 알림 1건씩(총 80만 건)을 알림발송내역에 적재한다. SQL 텍스트는 하나로 고정하고 조건값은 모두 바인드 변수로 넘기는데, 발송 함수가 반복문 안에서 커서를 매 회 새로 열고 닫는 구조라 실행마다 Parse 콜이 한 번씩 나간다. 8개의 발송 세션이 동시에 돌아 응답이 64초로 늘고 대기 이벤트 상위에 library cache: mutex(라이브러리 캐시) 경합이 잡혔다. 아래는 이 배치 세션의 비재귀·재귀 statement를 나눠 집계한 TKPROF이다. 지연의 원인을 직접 지목한 것으로 옳은 것은? (단, 옵티마이저 통계는 최신이고 SQL 텍스트는 대소문자·공백까지 동일해 라이브러리 캐시에서 공유되며, Shared Pool은 여유가 충분해 에이징으로 인한 강제 재파싱은 없다.)

| 아 래                                             |         |          |         |      |         |         |        |
|-------------------------------------------------|---------|----------|---------|------|---------|---------|--------|
| OVERALL TOTALS FOR ALL NON-RECURSIVE STATEMENTS |         |          |         |      |         |         |        |
| Call                                            | Count   | CPU Time | Elapsed | Disk | Query   | Current | Rows   |
| Parse                                           | 800000  | 20.000   | 55.000  | 0    | 0       | 0       | 0      |
| Execute                                         | 800000  | 8.000    | 9.000   | 0    | 1600000 | 900000  | 800000 |
| Fetch                                           | 0       | 0.000    | 0.000   | 0    | 0       | 0       | 0      |
| Total                                           | 1600000 | 28.000   | 64.000  | 0    | 1600000 | 900000  | 800000 |
| Misses in library cache during parse: 12        |         |          |         |      |         |         |        |
| OVERALL TOTALS FOR ALL RECURSIVE STATEMENTS     |         |          |         |      |         |         |        |
| Call                                            | Count   | CPU Time | Elapsed | Disk | Query   | Current | Rows   |
| Parse                                           | 20      | 0.010    | 0.010   | 0    | 0       | 0       | 0      |
| Execute                                         | 40      | 0.020    | 0.020   | 0    | 120     | 0       | 0      |
| Fetch                                           | 60      | 0.010    | 0.010   | 0    | 180     | 0       | 40     |
| Total                                           | 120     | 0.040    | 0.040   | 0    | 300     | 0       | 40     |
| Misses in library cache during parse: 8         |         |          |         |      |         |         |        |

- ① 비재귀 Parse가 CPU 20.0초로 Total CPU 28.0초의 대부분을 차지하고 라이브러리 캐시 미스도 비재귀 12건·재귀 8건이 잡혔으니, 병목은 계획을 새로 만드는 하드파싱이다. cursor\_sharing=FORCE로 하드파싱을 소프트파싱으로 돌리면 라이브러리 캐시 경합과 Parse elapsed 55.0초가 함께 줄어든다.
- ② 비재귀 User Call이 Parse 80만 + Execute 80만 = 160만 콜이고 Fetch는 0이라, 64.0초를 160만으로 나누면 콜당 40마이크로초로 정체는 클라이언트-서버 왕복 자체에 있다. 8개 세션이 동시에 던지는 이 왕복을 배열 처리와 커넥션 풀링으로 줄이면 Parse elapsed 55.0초의 대기가 해소된다.
- ③ 라이브러리 캐시 미스는 12건뿐이라 하드파싱은 사실상 없고, 병목은 Parse 콜 80만 건(Parse:Execute = 1:1)이 라이브러리 캐시 mutex를 다투는 소프트파싱 폭주다. 커서를 한 번만 파싱해 반복 실행(session\_cached\_cursors·홀드 커서)하도록 바꿔 Parse 콜을 줄이면 64초가 해소된다.
- ④ Execute가 Query 1,600,000·Current 900,000으로 함께 250만 블록을 만지며 80만 행을 적재해 행당 3블록이 넘고, 8개 세션이 이 논리 읽기를 동시에 일으켜 버퍼 캐시 경합을 만든 것이 64초의 골격이다. DB 버퍼 캐시를 키워 이 논리 읽기를 줄이면 Parse 단계의 대기도 함께 사라진다.

4

**DBMS\_XPLAN 출력 하단의 Note 절에 찍히는 두 문구 — 동적 샘플링(dynamic sampling)과 카디널리티 피드백(cardinality feedback) — 이 그 계획을 예상 계획으로 볼지 실제 계획으로 볼지에 대해 갖는 의미로 가장 적절한 것은?**

- ① Note에 "dynamic statistics used: dynamic sampling"이 찍혔다면, 이는 옵티마이저가 파싱 시점에 대상 객체의 통계가 부족하다고 보고 블록 일부를 표본 조사해 카디널리티 추정을 보정했다는 뜻일 뿐, 그 계획은 여전히 실측이 아닌 추정 계획이다.
- ② Note에 "cardinality feedback used for this statement"가 보이면 옵티마이저가 그 문장을 실제로 수행하며 각 단계의 실측 행 수를 계획에 채워 넣은 것이므로, 그 출력은 예상 계획이 아니라 단계별 실측 행 수가 담긴 실제 실행계획으로 읽어야 한다.
- ③ EXPLAIN PLAN으로 적재한 계획을 DBMS\_XPLAN.DISPLAY로 볼 때도, 문장을 한 번도 수행하지 않았지만 Note에 카디널리티 피드백이 적용됐다는 문구가 함께 출력되어 재최적화된 계획을 미리 확인할 수 있다.
- ④ 동적 샘플링은 대상 문장을 실제로 끝까지 수행하면서 반환된 행을 세어 통계를 만드는 기능이므로, Note에 이 문구가 있으면 그 계획은 수행이 완료된 뒤 얻은 실제 실행계획으로 보아야 한다.

5

**간편구독(PG) 거래 정산에서 구독료내역(약 1억 5,000만 건)을 구독자(약 600만 건), 콘텐츠목록(약 4만 건)와 조인해 구독자·콘텐츠카테고리별 집계 400만 행을 뽑는 SQL의 TKPROF이다. 구독료내역 Full Scan은 멀티블록이라 금방 끝날 것으로 봤는데 응답이 45초를 넘겨 느리다는 신고가 들어왔다. 아래 트레이스에 대한 해석으로 가장 적절한 것은? (단, 옵티마이저 통계는 최신이고 CPU·디스크 자원의 외부 영향은 없으며, 세 테이블에 인덱스가 없어 Full Scan + Hash Join으로만 수행된다.)**

| 아 래       |                                                                    |          |              |        |         |         |         |
|-----------|--------------------------------------------------------------------|----------|--------------|--------|---------|---------|---------|
| Call      | Count                                                              | CPU Time | Elapsed Time | Disk   | Query   | Current | Rows    |
| Parse     | 1                                                                  | 0.010    | 0.010        | 0      | 0       | 0       | 0       |
| Execute   | 1                                                                  | 0.000    | 0.000        | 0      | 0       | 0       | 0       |
| Fetch     | 4000                                                               | 8.000    | 45.000       | 240000 | 3000000 | 0       | 4000000 |
| Total     | 4002                                                               | 8.010    | 45.010       | 240000 | 3000000 | 0       | 4000000 |
| Rows      | Row Source Operation                                               |          |              |        |         |         |         |
| 4000000   | HASH GROUP BY (cr=3000000 pr=240000 pw=150000 time=44000000 us)    |          |              |        |         |         |         |
| 150000000 | HASH JOIN (cr=3000000 pr=90000 pw=0 time=20000000 us)              |          |              |        |         |         |         |
| 40000     | TABLE ACCESS FULL 콘텐츠목록 (cr=4000 pr=0 pw=0 time=50000 us)          |          |              |        |         |         |         |
| 150000000 | HASH JOIN (cr=2996000 pr=90000 pw=0 time=18000000 us)              |          |              |        |         |         |         |
| 6000000   | TABLE ACCESS FULL 구독자 (cr=96000 pr=6000 pw=0 time=1200000 us)      |          |              |        |         |         |         |
| 150000000 | TABLE ACCESS FULL 구독료내역 (cr=2900000 pr=84000 pw=0 time=8000000 us) |          |              |        |         |         |         |

- ① Fetch 한 줄에 CPU 8.000초·Elapsed 45.000초가 잡혔고 4,000회 Fetch로 400만 행을 퍼 올리며 논리 읽기 3,000,000블록을 훑었으니, 블록마다 해시 연산을 돌리는 CPU가 응답의 뼈대다. 인덱스 없이 Full Scan + Hash Join으로만 도는 구조이므로, 병렬도(DOP)를 높여 이 CPU를 병렬 서버로 나눠 주면 45초가 개선된다.
- ② 논리 I/O 300만 중 24만(8%)이 디스크라 BCHR은 92.0%인데, 물리 읽기 240,000의 62.5%(150,000)는 HASH GROUP BY가 400만 그룹을 해시 영역에 다 담지 못해 temp로 흘린 spill이다(pw=150,000). 병목은 구독료내역 스캔이 아니라 집계의 디스크 spill이므로, PGA(해시/정렬 영역)를 키워 spill을 없애야 한다.
- ③ Row Source의 cr 3,000,000 중 구독료내역 Full Scan이 2,900,000(96.7%)이고 물리 읽기도 pr=84,000으로 세 스캔 가운데 가장 크니, 1억 5,000만 건을 통째로 훑는 이 노드가 45초의 정체다. 구독일시에 인덱스를 만들어 이 Full Scan을 Index Range Scan으로 바꾸면 cr 290만이 줄면서 응답이 해소된다.
- ④ 안쪽 HASH JOIN(cr=2,996,000 pr=90,000)이 구독자 600만 행을 build로 잡고 구독료내역 1억 5,000만 건을 probe하느라 해시 영역을 넘겨 물리 읽기 240,000을 만든 것이므로, 4만 건짜리 콘텐츠목록(cr=4,000)을 build로 올리도록 조인 순서를 바꿔 해시 영역에 다 들어가게 하면 spill과 37초 대기가 사라진다.

소방 안전점검 관제 시스템의 점검내역 테이블(약 940만 건)에는 결합 인덱스 점검\_IX = (관할서코드, 점검일시)가 있다. 관할 소방서명을 함께 얻기 위해 소방서 테이블(기본키 소방서\_PK = 관할서코드)과 조인하는 다음 SQL의 실행계획이다. 이 실행계획에 대한 설명으로 가장 적절한 것은?

## 아 래

```
SELECT k.점검내역ID, k.점검일시, k.대상건물명, f.소방서명
FROM 점검내역 k, 소방서 f
WHERE k.관할서코드 = 'S07'
      AND k.점검일시 >= DATE '2026-06-01'
      AND k.점검일시 < DATE '2026-07-01'
      AND k.판정결과 = '부적합'
      AND f.관할서코드 = k.관할서코드;
```

| Id | Pid | Operation                        | ( Cost | Card  | Bytes )  |
|----|-----|----------------------------------|--------|-------|----------|
| 0  | -   | SELECT STATEMENT                 | ( 418  | 3600  | 190800 ) |
| 1  | 0   | NESTED LOOPS                     | ( 418  | 3600  | 190800 ) |
| 2  | 1   | TABLE ACCESS BY INDEX ROWID 점검내역 | ( 412  | 3600  | 126000 ) |
| 3  | 2   | INDEX RANGE SCAN 점검_IX           | ( 29   | 12000 | )        |
| 4  | 1   | TABLE ACCESS BY INDEX ROWID 소방서  | ( 1    | 1     | 18 )     |
| 5  | 4   | INDEX UNIQUE SCAN 소방서_PK         | ( 0    | 1     | )        |

- ① 점검\_IX의 INDEX RANGE SCAN이 Cost 29로 훑어 낸 Card 12,000이 이 SQL의 최종 결과 건수이며, 그 위 TABLE ACCESS BY INDEX ROWID 점검내역(Cost 412)은 ROWID로 대상건물명 등 나머지 컬럼을 채우는 단계라 건수 감소 없이 12,000건을 그대로 NESTED LOOPS로 넘긴다.
- ② 관할서코드(등치)와 점검일시(범위)까지가 점검\_IX의 인덱스 액세스 조건으로 INDEX RANGE SCAN에서 처리(Card 12,000)되고, 인덱스에 없는 판정결과는 TABLE ACCESS BY INDEX ROWID 단계 필터로 걸러져 Card가 3,600으로 줄어든다.
- ③ WHERE의 세 조건(관할서코드 등치·점검일시 범위·판정결과 등치)이 모두 점검\_IX의 인덱스 액세스 조건이 되어 Cost 29의 INDEX RANGE SCAN 단계에서 함께 스캔 시작·종료점을 정하며, Id 2의 Card 3,600은 소방서와의 NESTED LOOPS에서 걸러진 건수다.
- ④ 점검일시가 범위(>=, <) 조건이라 점검\_IX가 액세스 조건을 잃으므로 선두 칼럼 관할서코드 = 'S07' 등치까지 스캔 후 필터로 밀려나며, Id 3의 Card 12,000은 인덱스를 처음부터 끝까지 훑은 뒤 필터로 남은 건수이고 Cost 29가 그 전수 스캔 비용이다.

7

온라인 쇼핑몰의 상품평 시스템에서 상품평 테이블(약 5,000만 건, 블록당 약 125행 → 약 40만 블록)을 특정 상품카테고리 조건으로 조회한다. 카테고리 컬럼의 인덱스 상품평\_N1로 30만 건을 뽑는데, 인덱스를 타는데도 30초가 걸린다. 아래 TKPROF에서 지연의 원인을 직접 지목한 것으로 옳은 것은?

| 아 래     |                                                                            |          |              |       |        |         |        |
|---------|----------------------------------------------------------------------------|----------|--------------|-------|--------|---------|--------|
| Call    | Count                                                                      | CPU Time | Elapsed Time | Disk  | Query  | Current | Rows   |
| Parse   | 1                                                                          | 0.005    | 0.005        | 0     | 0      | 0       | 0      |
| Execute | 1                                                                          | 0.000    | 0.000        | 0     | 0      | 0       | 0      |
| Fetch   | 501                                                                        | 4.000    | 29.995       | 60000 | 271500 | 0       | 300000 |
| Total   | 503                                                                        | 4.005    | 30.000       | 60000 | 271500 | 0       | 300000 |
| Rows    | Row Source Operation                                                       |          |              |       |        |         |        |
| 300000  | TABLE ACCESS BY INDEX ROWID 상품평 (cr=271500 pr=60000 pw=0 time=29400000 us) |          |              |       |        |         |        |
| 300000  | INDEX RANGE SCAN 상품평_N1 (cr=1500 pr=0 pw=0 time=420000 us)                 |          |              |       |        |         |        |

- ① 병목은 상품평\_N1의 INDEX RANGE SCAN이다. 5,000만 건 테이블에서 30만 건을 뽑느라 인덱스 리프를 cr 1,500블록만큼 따라가는데, 이 리프 체인이 조각나 있어 501회 Fetch 내내 대기가 쌓였고 그것이 Fetch Elapsed 29.995초의 대부분이다. 인덱스를 리빌드해 리프 밀도를 회복하면 30.000초가 크게 준다.
- ② Fetch가 501회 일어나 300,000행을 회당 약 600행씩 실어 나르는 구조이므로, 클라이언트-서버 왕복 501회가 Fetch의 CPU 4.000초와 Elapsed 29.995초 사이의 26초 격차를 만든 주범이다. 배열 인출 크기를 키워 Fetch 횟수를 줄이면 왕복 대기가 사라지면서 TABLE ACCESS BY INDEX ROWID의 time 29.4초도 함께 줄어든다.
- ③ Disk 60,000은 논리 읽기 271,500의 22.1%로, 단일 블록 물리 읽기 6만 회가 곧 디스크 대기 6만 번을 뜻하니 TABLE ACCESS BY INDEX ROWID의 time 29.4초를 만든 근본 원인이다. 버퍼 캐시를 키워 이 6만 블록을 메모리에서 받게 하면 물리 읽기가 논리 읽기로 바뀌고 대기가 사라져 응답이 최적화된다.
- ④ 테이블 액세스 자체 블록 I/O가  $271,500 - 1,500 = 270,000$ 으로 ROWID 수 300,000과 거의 같아 클러스터링 팩터가 최악이다. 이 270,000이 time 29.4초를 차지하며 단일 블록 랜덤 읽기로 손익분기점을 넘었으므로, 필요한 컬럼을 인덱스에 더한 커버링 인덱스로 테이블 액세스를 제거하는 것이 답이다.

8

리프가 곧 데이터인 클러스터형 인덱스(SQL Server)·IOT(오라클)에 만든 보조(2차) 인덱스의 동작과, 긴 행을 갈라 두는 오버플로에 대한 설명으로 가장 적절하지 않은 것은?

- ① 클러스터형 인덱스(IOT)로 구성된 테이블에 만든 보조 인덱스는 리프에 행의 물리적 저장 위치를 가리키는 RID(ROWID)를 담으므로, 보조 인덱스로 조회하면 힙 테이블에서처럼 그 RID를 따라 한 번에 행에 도달한다.
- ② 클러스터형 인덱스는 리프 레벨이 곧 정렬된 행 데이터여서 별도의 테이블(힙) 세그먼트를 두지 않으며, 클러스터 키에 등가 조건을 준 Index Unique Scan은 리프에 도달하는 것만으로 행 전체를 얻어 별도의 테이블 액세스가 없다.
- ③ 보조 인덱스는 행의 물리적 위치 대신 클러스터 키(PK) 값을 행 식별자로 저장하므로, 보조 인덱스에서 키를 찾은 뒤 다시 클러스터형 인덱스를 그 키로 타고 내려가야 행에 닿는 2단계 접근이 일어난다.
- ④ 오라클 IOT는 행이 지나치게 길면 지정한 임계 이상(또는 INCLUDING으로 지정한) 컬럼을 오버플로 세그먼트에 따로 저장하며, 그 컬럼을 읽어야 할 때는 리프에서 오버플로 세그먼트로 한 번 더 이동해야 한다.

9

여러 컬럼을 함께 조건으로 거는 조회를 위해, 두 컬럼을 묶은 결합(concatenated) 인덱스 하나로 설계할지 아니면 단일 컬럼 인덱스 여러 개를 두고 인덱스 병합(index merge)에 맡길지 판단하는 설계에 대한 설명으로 가장 적절하지 않은 것은?

- ① 두 컬럼에 모두 등가 조건이 자주 걸린다면, 선택도가 높은 컬럼을 선두에 둔 결합 인덱스 하나가 스캔 시작·종료 지점을 좁게 잡아 훑는 리프 블록을 줄이므로, 단일 컬럼 인덱스 두 개를 각각 두는 것보다 대체로 효율적이다.
- ② 같은 컬럼 집합이라도 컬럼 순서는 조회 패턴을 따라야 하며, 등가 조건 컬럼을 앞에 두고 범위 조건 컬럼을 뒤에 두어야 선행 컬럼에서 스캔 구간을 좁힌 뒤 후행 범위 조건을 이어 갈 수 있다.
- ③ 인덱스를 늘리면 조회가 고를 선택지는 늘지만 컬럼 수가 많아질수록 리프 엔트리가 길어지고 DML마다 갱신할 인덱스가 늘어 부하가 커지므로, 흩어진 단일 컬럼 인덱스를 결합 인덱스로 묶어 개수를 줄이는 편이 DML 부하 관리에 유리한 경우가 많다.
- ④ 옵티마이저가 단일 컬럼 인덱스 두 개를 병합(index merge)해 두 조건의 교집합을 구할 수 있으므로, 두 컬럼을 함께 거는 조회에서는 개별 인덱스 두 개의 병합이 결합 인덱스 하나를 온전히 대신한다. 따라서 결합 인덱스를 굳이 따로 설계할 필요는 없다.

10

최근에 접종 기록이 있는 대상자만 골라내기 위해, 접종대상 테이블(약 120만 건)에 대해 접종기록 테이블(약 4,100만 건)로 EXISTS 서브쿼리를 건 SQL의 실행계획이다. 상관 조건은 대상자ID 등치(=)이고, 접종일자 범위는 서브쿼리 안의 조건이다. 이 플랜에서 서브쿼리가 어떻게 처리되고 있는지에 대한 설명으로 옳은 것은?

아 래

```
SELECT c.대상자ID, c.대상자명, c.동주소
FROM 접종대상 c
WHERE EXISTS ( SELECT 1
                FROM 접종기록 v
                WHERE v.대상자ID = c.대상자ID
                  AND v.접종일자 >= DATE '2026-06-01' );
```

| Id  | Operation                   | Name     |
|-----|-----------------------------|----------|
| 0   | SELECT STATEMENT            |          |
| * 1 | FILTER                      |          |
| 2   | TABLE ACCESS FULL           | 접종대상     |
| * 3 | TABLE ACCESS BY INDEX ROWID | 접종기록     |
| * 4 | INDEX RANGE SCAN            | 접종기록_IX1 |

- ① 상관 조건이 대상자ID 등치여서 서브쿼리가 세미 조인으로 Unnesting된 결과 Id 1이 HASH JOIN SEMI다. 접종대상 120만 건과 접종기록 4,100만 건을 해시 조인 한 번으로 매칭하므로, Id 3~4의 접종기록 액세스는 구동 행마다 반복되지 않는다.
- ② 서브쿼리가 Unnesting되지 않아 Id 1 FILTER로 남았고, 구동 테이블 접종대상(Id 2)의 각 행마다 서브쿼리(Id 3~4)를 반복 수행하되, 이미 확인한 대상자ID 값은 FILTER의 입력값 캐싱으로 재수행을 건너뛴다.
- ③ Id 1은 NOT EXISTS가 변환된 HASH JOIN ANTI이므로, Id 3~4에서 접종일자가 2026-06-01 이후인 행이 하나라도 잡히는 대상자를 걸러 내고, 접종대상 120만 건 가운데 최근 접종 기록이 없는 쪽만 결과로 돌려준다.
- ④ Id 1이 FILTER인 것은 맞지만 접종대상(Id 2)의 각 행마다 서브쿼리를 도는 것은 아니다. Id 4 INDEX RANGE SCAN 접종기록\_IX로 4,100만 건에서 접종일자 조건에 맞는 대상자ID를 미리 뽑아 해시 테이블에 적재한 뒤 조인 한 번으로 매칭을 끝낸다.

11

**소트 머지 조인(Sort Merge Join)과 NL 조인·해시 조인 사이의 선택 기준에 대한 설명으로 가장 적절한 것은?**

- ① 조인 키에 인덱스가 있어 양쪽 집합을 이미 정렬된 순서로 읽어 올 수 있으면, 소트 머지 조인은 값비싼 정렬(Sort Join) 단계를 건너뛰고 병합만 수행하므로, 이럴 때는 해시 테이블을 새로 만들어야 하는 해시 조인보다 유리해질 수 있다.
- ② 해시 조인은 조인 키를 해시 버킷에 나눠 같은 버킷끼리만 맞춰 보므로 부등호·범위 같은 비등치 조인을 처리하지 못한다. 이때는 양쪽을 조인 키로 정렬해 순서대로 비교하는 소트 머지 조인만이 대안이고, NL 조인은 인덱스로 상대를 찾지 못해 비등치 조인을 아예 수행할 수 없다.
- ③ 소트 머지 조인은 병합 전에 Sort Join 단계에서 양쪽 집합을 조인 키로 정렬해 두고 그 순서를 결과까지 이어가므로, 조인 키가 아닌 다른 컬럼으로 ORDER BY를 걸어도 이미 정렬된 스트림을 재사용해 추가 정렬 없이 함께 해결되고 뒤따르는 SORT ORDER BY가 사라진다.
- ④ 해시 조인은 Build Input으로 해시 테이블을 만들어야 해서 work area(PGA)를 크게 소비하지만, 소트 머지 조인의 Sort Join은 양쪽을 읽으며 순서대로 맞춰 가는 스트리밍 처리라 메모리를 쓰지 않는다. 따라서 가용 메모리가 부족한 환경에서는 소트 머지 조인이 대안이 된다.

12

**옵티마이저의 쿼리 변환에서 복합 뷰 Merging과 서브쿼리 Unnesting이 서로 맞물려 동작하는 방식에 대한 설명으로 가장 적절하지 않은 것은?**

- ① 복합 뷰 Merging은 두 쿼리 블록을 하나로 합쳐 조인·액세스 순서의 선택 폭을 넓히는 변환이므로, 병합이 문법적으로 가능하지만 하면 옵티마이저는 비용 비교 없이 그 뷰를 병합한다.
- ② GROUP BY나 DISTINCT를 담은 복합 뷰의 병합은 단순 뷰 병합과 달리 비용 기반으로 판단되어, 병합이 오히려 불리하다고 평가되면 옵티마이저는 그 뷰를 개별 수행한다.
- ③ 뷰가 병합되지 못하고 독립된 블록으로 남더라도, 옵티마이저는 바깥 조건을 뷰 안으로 밀어 넣는 조건절 Pushing을 시도해 뷰가 스캔·처리할 행을 미리 줄일 수 있다.
- ④ 상관 서브쿼리가 Unnesting되어 조인으로 풀리면 그 블록에 대해 조인 순서·방식을 비용으로 고르는 선택지가 새로 생기는 식으로, 한 변환의 성사가 뒤이은 최적화의 여지를 넓히기도 한다.

13

**IT 헬프데스크 대시보드가 지원티켓 테이블(약 3,800만 건)에서 상태·우선순위 조합별 건수를 실시간으로 조회한다. 애플리케이션이 상태·우선순위 값을 바인드 없이 리터럴로 결합해 SQL을 발행하는 탓에, 상태(6종)×우선순위(5종) = 30가지 조합만큼 서로 다른 SQL 텍스트가 라이브러리 캐시에 쌓이고 대기 이벤트 상위에 library cache: mutex X가 잡혔다. DBA는 CURSOR\_SHARING 조정을 검토 중이다. 커서 공유와 CURSOR\_SHARING에 대한 설명으로 가장 적절하지 않은 것은?**

아래

```
-- 애플리케이션이 상태·우선순위를 리터럴로 결합해 발행 (바인드 미사용)
SELECT COUNT(*) FROM 지원티켓 WHERE 상태 = 'OPEN' AND 우선순위 = 1;
SELECT COUNT(*) FROM 지원티켓 WHERE 상태 = 'OPEN' AND 우선순위 = 2;
SELECT COUNT(*) FROM 지원티켓 WHERE 상태 = 'HOLD' AND 우선순위 = 3;
-- ... 상태(6)×우선순위(5) = 30가지 조합마다 SQL_ID가 갈려 하드파싱 반복

-- 검토 중인 조치
ALTER SESSION SET CURSOR_SHARING = FORCE; -- 기본값은 EXACT
```

- ① 기본(EXACT) 모드에서는 리터럴만 다른 SQL이 각각 별도 부모 커서로 하드파싱되어 라이브러리 캐시에 30개의 커서가 쌓이고, 파싱 부하와 library cache: mutex 경합이 커진다.
- ② CURSOR\_SHARING = FORCE로 바꾸면 오라클이 리터럴을 시스템 생성 바인드(:SYS\_B\_0 등)로 치환해 30가지 텍스트를 하나로 만들어 커서를 공유하므로, 반복 하드파싱이 줄어든다.
- ③ 기본(EXACT) 모드도 실행 전에 SQL 텍스트의 공백·대소문자·주석을 내부적으로 정규화해 비교하므로, 리터럴 값만 같다면 들여쓰기나 대소문자가 달라도 같은 부모 커서로 묶여 하드파싱이 일어나지 않는다.
- ④ FORCE는 리터럴을 바인드로 치환하면서 최초 실행 때 바인드 피킹된 계획 하나를 공유하므로, 값에 따라 최적 계획이 갈리는(분포가 치우친) 컬럼에서는 일부 실행에 불리한 계획이 재사용될 수 있다.



14

옵티마이저 통계의 수집·관리 방식(DBMS\_STATS 수집, 샘플 크기, 자동 통계 수집, stale, 동적 샘플링)에 대한 설명으로 가장 적절한 것은?

- ① 동적 샘플링(dynamic sampling)은 통계가 없거나 오래된 객체를 파싱 시점에 훑어 즉석 통계를 만드는 기능이므로, 동적 샘플링 레벨을 높여 두면 DBMS\_STATS로 미리 수집해 둔 통계의 stale 상태까지 자동으로 해소된다.
- ② DBMS\_STATS로 통계를 수집할 때 ESTIMATE\_PERCENT를 AUTO\_SAMPLE\_SIZE로 두면 옵티마이저가 대상 객체 특성에 맞춰 샘플 크기를 스스로 정하므로, 전수 조사보다 훨씬 적은 비용으로도 실용적 정확도의 통계를 얻을 수 있다.
- ③ 통계를 전수(100%)로 수집해 표본 오차를 없애는 것과 그 통계가 최신 상태로 유지되는 것은 같은 축이므로, ESTIMATE\_PERCENT를 100으로 잡아 두면 이후 데이터가 대량 변경돼도 통계가 stale로 표시되지 않는다.
- ④ 자동 통계 수집 작업(auto stats job)이 stale 통계를 갱신하는 것과 그 테이블을 참조하는 커서가 새 통계로 다시 최적화되는 것은 하나의 단계이므로, 자동 작업이 통계를 갱신하는 순간 관련 커서의 실행계획도 즉시 새것으로 교체된다.

15

장학금 지급 마감 배치가 학기별 심사 결과를 장학금지급 테이블에 반영한다. 아래 MERGE는 사정 결과(장학사정)를 대상(장학금지급)에 Upsert하되, 사정 상태가 '탈락'으로 갱신되는 행은 삭제까지 한 문장으로 처리한다. 장학금지급의 (학번, 학기)는 기본키다. 이 MERGE의 동작과 조건에 대한 설명으로 가장 적절하지 않은 것은?

아래

```

MERGE INTO 장학금지급 t
USING 장학사정 s
ON (t.학번 = s.학번 AND t.학기 = s.학기)
WHEN MATCHED THEN
    UPDATE SET t.지급액 = s.사정액, t.지급상태 = s.사정상태
    DELETE WHERE (s.사정상태 = '탈락')
WHEN NOT MATCHED THEN
    INSERT (학번, 학기, 지급액, 지급상태)
VALUES (s.학번, s.학기, s.사정액, s.사정상태);

```

- ① MERGE는 대상 t의 각 행에 대해 ON 조건(학번·학기 일치)으로 매칭을 판정해, 매칭이면 UPDATE(그리고 DELETE WHERE 충족 시 그 행 삭제)를, 미매칭인 소스 행은 INSERT를 한 문장으로 수행한다.
- ② WHEN MATCHED의 DELETE WHERE는 ON 조건으로 이미 매칭돼 UPDATE된 행에만 적용되며, 갱신된 뒤의 값을 기준으로 평가된다 — 매칭되지 않은 t의 기존 행은 이 절로 삭제되지 않는다.
- ③ DELETE WHERE는 ON 조건과 무관하게 대상 테이블 전체를 훑어, UPDATE 이전의 원래 값을 기준으로 조건을 만족하면 매칭되지 않은 행까지 함께 삭제한다.
- ④ 소스(장학사정)에 같은 (학번, 학기)가 중복으로 들어와 대상 한 행에 두 소스 행이 매칭되면, 어느 값으로 UPDATE할지 확정할 수 없어 MERGE는 런타임 오류(ORA-30926)로 실패한다.



국가 가축이력 시스템의 가축개체 테이블은 시도코드(2자리 행정구역 코드)를 기준으로 아래처럼 LIST 파티셔닝되어 있다. 각 선택지는 SELECT COUNT(\*) FROM 가축개체 WHERE ...의 WHERE 절과 그에 대한 Partition Pruning 판정이다. 옵티마이저의 pruning 동작을 옳게 설명한 것으로 가장 적절한 것은?

## 아 래

```
CREATE TABLE 가축개체 (
  개체번호    NUMBER,
  축종코드    VARCHAR2(3),
  시도코드    VARCHAR2(2),
  등록일자    DATE
)
PARTITION BY LIST (시도코드) (
  PARTITION p_capital VALUES ('11', '28', '41'),    -- 수도권(서울 · 인천 · 경기)
  PARTITION p_yeongnam VALUES ('26', '27', '31', '48'), -- 영남권
  PARTITION p_honam  VALUES ('29', '45', '46'),    -- 호남권
  PARTITION p_etc     VALUES (DEFAULT)             -- 그 외 전부
);
```

- ① WHERE 시도코드 = '27' — '27'이 정의된 파티션(p\_yeongnam) 하나만 접근하고 나머지 세 파티션은 건너뛰는다.
- ② WHERE 시도코드 IN ('11', '26') — 두 값이 서로 다른 파티션에 속하므로, LIST 파티션은 IN 조건을 경계와 대조하지 못해 전 파티션을 스캔한다.
- ③ WHERE SUBSTR(시도코드, 1, 2) = '41' — 시도코드가 이미 2자리라 SUBSTR 결과가 원본과 같으므로, 옵티마이저가 이를 파티션 키 값으로 그대로 인식해 p\_capital 한 파티션만 pruning한다.
- ④ WHERE 시도코드 <> '11' — '11'이 든 p\_capital만 빼면 되므로, 옵티마이저가 나머지 세 파티션만 정확히 pruning한다.

상수도 요금 정산 야간 배치가 급수사용량(약 4억 2천만 건)과 급수전(약 5,100만 건)을 급수전번호로 조인한 뒤, 지역코드별로 사용량을 집계한다. 두 테이블은 어느 쪽도 급수전번호로 파티셔닝되어 있지 않다. 아래 병렬 실행계획에서 조인과 집계가 병렬 서버(Slave) 사이에 어떻게 분배되는지에 대한 설명으로 가장 적절한 것은?

| 아래  |                     |          |       |        |
|-----|---------------------|----------|-------|--------|
| Id  | Operation           | Name     | TQ    | IN-OUT |
| 0   | SELECT STATEMENT    |          |       |        |
| 1   | PX COORDINATOR      |          |       |        |
| 2   | PX SEND QC (RANDOM) | :TQ10003 | Q1,03 | P->S   |
| 3   | HASH GROUP BY       |          | Q1,03 | PCWP   |
| 4   | PX RECEIVE          |          | Q1,03 | PCWP   |
| 5   | PX SEND HASH        | :TQ10002 | Q1,02 | P->P   |
| 6   | HASH GROUP BY       |          | Q1,02 | PCWP   |
| * 7 | HASH JOIN           |          | Q1,02 | PCWP   |
| 8   | PX RECEIVE          |          | Q1,02 | PCWP   |
| 9   | PX SEND HASH        | :TQ10000 | Q1,00 | P->P   |
| 10  | PX BLOCK ITERATOR   |          | Q1,00 | PCWC   |
| 11  | TABLE ACCESS FULL   | 급수전      | Q1,00 | PCWP   |
| 12  | PX RECEIVE          |          | Q1,02 | PCWP   |
| 13  | PX SEND HASH        | :TQ10001 | Q1,01 | P->P   |
| 14  | PX BLOCK ITERATOR   |          | Q1,01 | PCWC   |
| 15  | TABLE ACCESS FULL   | 급수사용량    | Q1,01 | PCWP   |

- ① 5,100만 건인 소형 급수전(Id 11)을 :TQ10000으로 병렬 서버 전체에 복제(broadcast)하고, 4억 2천만 건인 급수사용량(Id 15)은 PX BLOCK ITERATOR로 자기 블록만 읽어 재분배 없이 probe하는 broadcast 조인이다. 각 서버가 조인 결과를 이미 쥐고 있어 Id 6과 Id 3의 집계는 재분배 없이 서버 로컬로 끝난다.
- ② 조인 입력 급수전(Id 9)·급수사용량(Id 13)을 조인 키(급수전번호) 해시로 각각 재분배(PX SEND HASH, TQ10000·TQ10001)해 hash-hash로 조인하고, 그와 별개로 집계는 조인 뒤 부분 집계(Id 6) → 그룹 키(지역코드) 해시로 재분배(Id 5) → 최종 집계(Id 3)로, 조인 재분배와 다른 TQ에서 일어난다.
- ③ 급수전과 급수사용량이 급수전번호로 동일하게 파티셔닝돼 있어 Id 7 HASH JOIN이 대응 파티션끼리 서버 하나씩 맡아 처리하는 full 파티션 와이즈 조인이고, Id 9·13의 큐는 파티션을 서버에 배정하는 통로일 뿐 행을 옮기지 않는다. 실제 재분배는 지역코드 집계를 위한 Id 5 :TQ10002에서만 일어난다.
- ④ 조인은 Id 9·13의 PX SEND HASH로 hash-hash 재분배되는 것이 맞지만, 조인 결과가 이미 지역코드로 뭉쳐 있어 집계는 재분배 없이 한 번의 HASH GROUP BY로 끝난다. Id 5 :TQ10002는 그 결과를 QC로 올리기 전에 거치는 통로이고, Id 6과 Id 3은 같은 집계를 중복 표기한 것일 뿐 서로 다른 단계가 아니다.

정렬이 사용하는 PGA work area(Sort Area)의 크기와 자동 PGA 관리(PGA\_AGGREGATE\_TARGET)에 대한 설명으로 가장 적절하지 않은 것은?

- ① 정렬은 PGA의 work area(Sort Area)에서 이뤄지며, SORT GROUP BY·SORT JOIN 같은 소트 오퍼레이션의 대상이 work area 안에 다 담기면 메모리 내 정렬(optimal)로 끝나 추가 디스크 I/O가 없다.
- ② 정렬 대상이 work area보다 크면 일부를 Temp 테이블스페이스로 내렸다 다시 읽어 병합하는 디스크 소트가 되며, 한 번의 추가 패스로 끝나면 one-pass, 여러 번 나눠 병합하면 multi-pass가 되어 패스가 늘수록 I/O 부담이 커진다.
- ③ 자동 PGA 관리에서는 PGA\_AGGREGATE\_TARGET으로 인스턴스 전체 PGA 목표치를 정해 두면 각 세션의 work area 크기를 서버가 자동으로 조절하므로, 예전처럼 SORT\_AREA\_SIZE를 세션마다 수동으로 잡지 않아도 된다.
- ④ PGA\_AGGREGATE\_TARGET은 인스턴스가 쓸 PGA 총량의 목표치이므로, 동시에 수행되는 정렬 하나가 그 목표치 전부를 자신의 work area로 통째로 할당받아 정렬에 쓸 수 있다.

19

기부금 모금 관리 시스템의 모금대장에는 캠페인 A·B·C의 누적 모금액이 각각 100,000·200,000·300,000원(합 600,000)으로 들어 있다. 집계 세션 TX2가 t1에 SELECT SUM(누적액) FROM 모금대장을 시작해 블록을 A→B→C 순으로 훑는 사이, 다른 세션 TX1이 t2에 B에 50,000을, t3에 C에 80,000을 적립하고 각각 즉시 COMMIT한다. TX2의 집계는 t4에 끝난다. t4에 TX2의 SELECT가 반환하는 SUM(누적액)으로 적절한 것은? (단, 오라클이며 자동 Undo 관리로 관련 Undo가 덮어써지지 않았고, 격리수준은 기본값 READ COMMITTED이다.)

아래

| 시각                                          | TX1 (적립 · 커밋)                                                             | TX2 (집계 SELECT, t1 시작)                              |
|---------------------------------------------|---------------------------------------------------------------------------|-----------------------------------------------------|
| t1                                          |                                                                           | SELECT SUM(누적액) FROM 모금대장;<br>→ 블록을 A→B→C 순으로 스캔 시작 |
| t2                                          | UPDATE 모금대장 SET 누적액=누적액+50000<br>WHERE 캠페인='B'; COMMIT; (B 200000→250000) |                                                     |
| t3                                          | UPDATE 모금대장 SET 누적액=누적액+80000<br>WHERE 캠페인='C'; COMMIT; (C 300000→380000) |                                                     |
| t4                                          |                                                                           | → SUM 집계 완료 · 반환                                    |
| 초기: A=100000, B=200000, C=300000 (합 600000) |                                                                           |                                                     |

- ① 730,000
- ② 650,000
- ③ 600,000
- ④ 680,000

20

제품 리콜 처리 시스템(SQL Server)에서 세션 58이 리콜대상 테이블(약 900만 건)의 특정 제조사 제품 약 6만 행을 한 UPDATE 문으로 갱신하자, 개별 행·페이지 잠금이 하나의 테이블(OBJECT) 잠금으로 대체됐다. 이때 sys.dm\_tran\_locks를 조회한 결과가 아래와 같다(RCSI 미사용, 기본 READ COMMITTED). 이 잠금 상태와 SQL Server의 잠금 에스컬레이션에 대한 해석으로 가장 적절하지 않은 것은?

아래

[ sys.dm\_tran\_locks - 리콜대상 대량 UPDATE 세션의 잠금 에스컬레이션 ]

| request_session_id | resource_type | request_mode | request_status |
|--------------------|---------------|--------------|----------------|
| 58                 | OBJECT        | X            | GRANT          |
| 72                 | OBJECT        | IS           | WAIT           |
| 80                 | OBJECT        | S            | WAIT           |

\* OBJECT = 테이블 수준 잠금, X = 배타, IS = 의도 공유, S = 공유  
\* GRANT = 획득 · 보유, WAIT = 요청 대기(블로킹됨)

- ① 세션 58이 리콜대상 테이블에 OBJECT 수준 X 잠금을 GRANT로 보유하므로, 같은 테이블에 IS·S를 요청한 세션 72·80은 X와 호환되지 않아 WAIT로 블로킹된다.
- ② 잠금 에스컬레이션은 한 문장이 대략 5,000개 이상의 행·페이지 잠금을 확보하면 이를 하나의 테이블(OBJECT) 잠금으로 대체해, 잠금 관리 메모리와 오버헤드를 줄이는 최적화다.
- ③ 테이블 잠금으로 대체되면 개별 행·페이지 잠금은 해제되고 굵은 잠금 하나만 남으므로, 잠금 개수는 줄지만 그 테이블에 대한 다른 세션의 동시성은 낮아진다.
- ④ SQL Server의 잠금 에스컬레이션은 테이블 잠금에서 출발해 경합이 감지되면 페이지→행 잠금으로 더 잘게 쪼개 내려가므로, 대량 UPDATE가 도는 중에도 다른 세션의 행 잠금 요청은 블로킹되지 않는다.

## 정답 및 해설

### ■ 정답 모아보기

|       |       |       |       |       |
|-------|-------|-------|-------|-------|
| 1. ④  | 2. ③  | 3. ③  | 4. ①  | 5. ②  |
| 6. ②  | 7. ④  | 8. ①  | 9. ④  | 10. ② |
| 11. ① | 12. ① | 13. ③ | 14. ② | 15. ③ |
| 16. ① | 17. ② | 18. ④ | 19. ③ | 20. ④ |

### 문제 1. 정답 ④

행 이전(Row Migration)이 조회에 비용을 더하는 이유는 바로 인덱스가 예전 위치를 그대로 가리킨 채 남아 있기 때문입니다. 행은 옮겨 가지만 ROWID는 바뀌지 않습니다.

행 이전(Row Migration) 발생 시

원래 블록 : 행 실체는 사라지고 '옮겨 간 위치' 포인터만 남음

새 블록 : 행 실체가 이동해 자리 잡음

ROWID : 원래 값 그대로 유지 → 인덱스는 갱신되지 않음

인덱스 경유 액세스 : 원래 블록 방문 → 포인터를 따라 새 블록 방문  
→ 블록 방문이 한 번 더 늘어 추가 I/O 발생

④는 이 동작을 정반대로 뒤집었습니다. 행 이전이 일어나도 인덱스의 ROWID는 갱신되지 않고 원래 값을 유지하며, 원래 블록에는 옮겨 간 곳을 가리키는 포인터만 남습니다. 그래서 인덱스로 그 행을 찾으면 원래 블록을 먼저 방문한 뒤 포인터를 따라 새 블록을 한 번 더 방문해야 하고, 이 추가 방문이 곧 행 이전의 성능 부담입니다. "ROWID가 새 블록을 가리키도록 갱신되어 곧바로 도달한다"는 서술은 이 부담이 왜 생기는지를 통째로 지워 버린 반대 진술입니다.

①·②·③은 옳습니다. 세그먼트-익스텐트의 관계와 익스텐트가 서로 떨어져 놓일 수 있다는 점(①), 행 연쇄가 처음부터 여러 블록에 나뉘는 현상이라는 점(②), 행 이전이 행을 옮기고 원래 자리에 포인터만 남긴다는 점(③)이 모두 저장 구조의 실제 동작에 부합합니다.

| 선택지 | 판정 | 이유                                                                                                                          |
|-----|----|-----------------------------------------------------------------------------------------------------------------------------|
| ①   | ○  | 오브젝트 하나가 하나의 세그먼트에 대응하고, 세그먼트의 익스텐트들은 물리적으로 서로 떨어진 자리에 놓일 수 있습니다                                                            |
| ②   | ○  | 행 연쇄는 한 행이 블록 하나에 못 담겨 처음부터 여러 블록에 나뉘는 현상이라, 읽을 때 연쇄된 블록을 모두 방문해 블록 I/O가 늘어납니다                                              |
| ③   | ○  | 행 이전은 커진 행을 다른 블록으로 옮기고 원래 자리에는 옮겨 간 위치를 가리키는 포인터만 남기는 현상입니다                                                                |
| ④   | ×  | 행 이전에서 ROWID는 원래 값을 유지해 인덱스는 갱신되지 않고, 원래 블록의 포인터를 따라 새 블록을 한 번 더 방문해야 합니다. "ROWID가 갱신되어 곧바로 도달한다"는 것은 이 추가 I/O를 지운 반대 서술입니다 |

#### ■ 이 문제의 핵심

1. 세그먼트 ⊃ 익스텐트 ⊃ 블록이며, 한 세그먼트의 익스텐트들은 물리적으로 떨어져 있을 수 있습니다.
2. 행 연쇄는 처음부터 한 행이 여러 블록에 나뉘는 현상입니다.
3. 행 이전은 갱신으로 커진 행을 다른 블록으로 옮기고 원래 자리에 포인터만 남기는 현상입니다.
4. 행 이전 시 ROWID는 그대로 유지되어, 인덱스 액세스는 원래 블록→새 블록으로 한 번 더 방문하는 추가 I/O를 냅니다.

■ 한 줄 정리 행 이전이 일어나도 인덱스의 ROWID는 원래 값을 유지한 채 원래 블록에 포인터만 남으므로 인덱스 액세스에 추가 방문이 드는데, "ROWID가 새 블록을 가리키도록 갱신되어 곧바로 도달한다"는 ④는 이 부담을 정반대로 뒤집은 서술입니다.

### 문제 2. 정답 ③

대기 이벤트는 크게 "디스크에서 블록이 오기를 기다리는 I/O 대기"와 "이미 메모리에서 처리 중인 블록을 놓고 부딪혀 기다리는 경합성 대기"로 갈립니다. 이름이 비슷해 보여도 원인과 처방이 다릅니다.

I/O 대기(디스크가 느려서 기다림)

db file sequential read : 한 블록씩 읽는 Single Block I/O 대기(인덱스 경유 등)

db file scattered read : 여러 블록을 묶어 읽는 Multiblock I/O 대기(Full Scan 등)

경합성 대기(블록이 이미 처리 중이라 기다림)

buffer busy waits : 다른 세션이 그 블록을 부적합한 방식으로 쓰는 중  
 read by other session : 다른 세션이 그 블록을 디스크→캐시로 읽는 중 → 중복 읽기 회피

③은 **read by other session**을 정확히 짚었습니다. 내가 원하는 블록을 다른 세션이 마침 디스크에서 캐시로 올리는 중이면, 나까지 같은 블록을 따로 읽어 캐시에 중복 적재하는 대신 그 읽기가 끝나기를 기다립니다. 같은 블록에 요청이 몰릴 때 나타나는 경합성 대기라는 설명이 이 이벤트의 성격에 맞습니다.

①·②·④는 I/O 대기와 경합성 대기의 경계를 뭉개졌습니다. **db file sequential read**는 디스크 읽기 대기이지 경합 대기가 아니고(①), **buffer busy waits**는 캐시 미스 시의 디스크 읽기 대기가 아니라 이미 처리 중인 블록에 대한 경합이며(②), **buffer busy waits·read by other session**은 스토리지 지연이 아니라 세션 간 블록 경합에서 비롯됩니다(④). 세 진술 모두 두 부류의 원인을 뒤섞어 처방을 엉뚱한 쪽으로 돌립니다.

| 선택지 | 판정 | 이유                                                                                                               |
|-----|----|------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | <b>db file sequential read</b> 는 한 블록씩 읽는 Single Block I/O 대기입니다. 이를 블록 경합 대기로 뭉개 것으로, 이름이 가리키는 디스크 읽기가 실제 성격입니다 |
| ②   | ×  | <b>buffer busy waits</b> 는 캐시 미스 시 디스크 읽기를 기다리는 대기가 아니라, 이미 처리 중인 블록을 놓고 부딪혀 기다리는 경합입니다. 캐시를 키운다고 원인이 사라지지 않습니다  |
| ③   | ○  | <b>read by other session</b> 은 다른 세션이 같은 블록을 디스크→캐시로 읽는 중이라 중복 읽기를 피해 기다리는, 요청이 몰릴 때 나타나는 경합성 대기입니다              |
| ④   | ×  | <b>buffer busy waits·read by other session</b> 은 스토리지 지연이 아니라 세션 간 블록 경합에서 비롯되므로, 스토리지 교체가 근본 처방이 되지 않습니다        |

#### ▪ 이 문제의 핵심

1. **db file sequential/scattered read**는 디스크에서 블록이 오기를 기다리는 I/O 대기입니다.
2. **buffer busy waits·read by other session**은 이미 처리 중인 블록을 놓고 부딪히는 경합성 대기입니다.
3. **read by other session**은 다른 세션이 같은 블록을 읽는 중이라 중복 읽기를 피해 기다리는 대기입니다.
4. 두 부류는 원인과 처방이 다릅니다 — I/O 대기는 읽기 경로를, 경합성 대기는 몰림을 손봐야 합니다.

▪ 한 줄 정리 I/O 대기(**db file sequential/scattered read**)와 경합성 대기(**buffer busy waits·read by other session**)는 원인이 다른데, **read by other session**을 "다른 세션의 읽기가 끝나기를 기다리는 경합성 대기"로 정확히 짚은 ③만 두 부류의 경계를 지켰습니다.

### 문제 3. 정답 ③

SQL 텍스트가 하나로 고정돼 있고 바인드 변수를 쓰므로, 커서는 라이브러리 캐시에서 공유됩니다. 그래서 파싱은 대부분 소프트파싱입니다. 먼저 하드파싱을 셉니다.

하드파싱율 = 라이브러리 캐시 미스(비재귀) ÷ 비재귀 Parse 콜  
 = 12 ÷ 800,000 ≈ 0.0015%  
 → 하드파싱은 사실상 없음(계획을 새로 만든 건 12회뿐)

재귀 statement도 Parse 20·Execute 40으로 미미합니다. 하드파싱이 데이터 덱서너리를 뒤지는 재귀 폭주가 없다는 뜻이니, 계획 생성 측은 병목이 아닙니다. 그런데 응답은 64초입니다. 파싱 콜 횟수와 파싱 대기를 봅니다.

Parse 콜 : Execute 콜 = 800,000 : 800,000 = 1 : 1 → 커서 미보유, 실행마다 재파싱  
 Parse elapsed - Parse CPU = 55.0 - 20.0 = 35.0초 ← 라이브러리 캐시 대기(mutex)  
 Parse elapsed 비중 = 55.0 ÷ 64.0 × 100 ≈ 85.9%  
 실효업(Execute) elapsed = 9.0초 (전체의 14.1%)

Parse에 55초가 잡혔는데 그중 35초가 CPU가 아니라 대기입니다. 하드파싱이 12회뿐인데도 대기가 큰 이유는, 소프트파싱도 라이브러리 캐시에서 커서를 찾느라 mutex를 잡기 때문입니다. 여기서 두 측을 분리해야 합니다.

측 A: 하드파싱(계획 생성·라이브러리 캐시 MISS) — 12건(0.0015%) → 병목 아님  
 측 B: 파싱 콜 횟수(소프트파싱·라이브러리 캐시 latch/mutex) — 800,000건 → 35초 대기의 정체

동시 8세션이 매 실행마다 Parse 콜을 던지면, 하드파싱이 0에 가까워도 소프트파싱끼리 같은 커서의 mutex를 서로 다투 측 B가 독립적으로 병목이 된다.

처방은 커서를 한 번만 파싱해 반복 실행하도록(홀드 커서·**session\_cached\_cursors**) 바꿔 Parse 콜 자체를 없애는 것입니다. Parse 콜이 줄면 라이브러리 캐시 mutex 경합도 함께 사라집니다. ③이 하드파싱 측(계획 생성)과 파싱 콜 측(소프트파싱 빈도)을 옳게 분리했습니다.

| 선택지 | 판정 | 이유                                                                                                                                                                                              |
|-----|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | Parse CPU 20.0초와 미스 12·8건을 하드파싱 비용으로 본 것이 오류입니다. 비재귀 미스 12건은 Parse 콜 80만의 0.0015%뿐이라 계획 생성은 사실상 없고, SQL은 이미 바인드 변수라 <b>cursor_sharing=FORCE</b> 가 바꿀 리터럴도 없습니다. 파싱 콜 측을 하드파싱 측으로 오인한 별개 측 혼동입니다 |
| ②   | ×  | 콜 160만과 콜당 40마이크로초는 산술로는 맞지만, User Call 수는 파싱·실행 횟수일 뿐 왕복 지연이 병목이라는 뜻이 아닙니다. 이 배치는 INSERT라 Fetch가 0이고 대기는 Parse의 mutex(55.0-20.0=35초)에서 났으므로, 배열 처리·커넥션 풀링은 왕복 측을 겨냥할 뿐 파싱 콜 mutex를 줄이지 못합니다    |
| ③   | ○  | 하드파싱율 12/800,000 $\approx$ 0.0015%로 계획 생성 측은 무죄이고, Parse:Execute = 1:1의 소프트파싱 80만 콜이 라이브러리 캐시 mutex를 다뤄 Parse elapsed 55초 중 35초가 대기임을 짚어, 커서 홀드로 Parse 콜을 없애는 처방이 정확합니다                         |
| ④   | ×  | Query 1,600,000 + Current 900,000 = 250만 블록은 Disk 0의 논리 읽기라 이미 전부 메모리에서 처리됐고, Execute elapsed는 9.0초(14.1%)뿐입니다. 병목은 실행 I/O 측이 아니라 Parse의 mutex 대기 측이므로, 버퍼 캐시를 키워도 55초의 파싱 대기는 그대로입니다           |

#### ■ 이 문제의 핵심

1. 하드파싱율 = 라이브러리 캐시 미스 ÷ Parse 콜. 여기서는 12/800,000  $\approx$  0.0015%로, 바인드 변수 + 텍스트 고정이라 계획 생성(하드파싱)은 사실상 없습니다.
2. 하드파싱 측(계획 생성)과 파싱 콜 측(소프트파싱 빈도)은 별개입니다. 하드파싱이 0에 가까워도 소프트파싱 콜이 많으면 라이브러리 캐시 mutex를 다뤄 독립적으로 병목이 됩니다.
3. Parse:Execute = 1:1은 커서 미보유의 신호입니다. 실행마다 Parse 콜이 나가 80만 회의 소프트파싱이 발생했고, Parse elapsed 55초 중 35초가 mutex 대기입니다.
4. 처방은 **cursor\_sharing**·버퍼 캐시·커넥션 풀링이 아니라 Parse 콜 제거(커서 홀드·**session\_cached\_cursors**)입니다. 한 번 파싱해 반복 실행하면 mutex 경합이 사라집니다.
5. 측을 뒤섞으면 하드파싱 처방(①)·왕복 처방(②)·I/O 처방(④)으로 헛짚습니다.

■ 한 줄 정리 하드파싱율이 0.0015%(12/800,000)로 계획 생성 측은 무죄인데도 64초가 걸린 것은 Parse:Execute = 1:1의 소프트파싱 80만 콜이 라이브러리 캐시 mutex를 다뤄(Parse elapsed 55초 중 35초 대기) 파싱 콜 측이 독립적으로 병목이기 때문이며, 처방은 커서를 홀드해 Parse 콜을 없애는 것이다.

#### 문제 4. 정답 ①

Note에 찍히는 두 문구는 언제 무엇을 근거로 계획을 세웠는가를 알려 줄 뿐, 계획이 "예상"에서 "실측"으로 바뀌었다는 뜻이 아닙니다.

dynamic sampling(동적 샘플링)

시점 : 파싱(하드 파싱) 시점

방법 : 통계가 부족한 객체의 블록 일부만 표본 조사 → 카디널리티 보정

성격 : 여전히 '추정' - 문장을 수행해 얻은 실측이 아님

cardinality feedback(카디널리티/통계 피드백)

시점 : 문장을 한 번 이상 '수행한 뒤', 다음 수행에서 재최적화

전제 : 실제 수행이 있어야 발동 → 수행 없는 EXPLAIN PLAN에는 나타나지 않음

성격 : 이전 수행의 실측을 반영해 다시 세운 계획이나, 화면의 행 수는 여전히 추정치

①은 동적 샘플링 Note를 정확히 해석했습니다. 통계가 부족한 객체를 파싱 시점에 표본 조사해 카디널리티를 보정했다는 표시일 뿐, 문장을 수행해 실측을 얻은 것이 아니므로 그 계획은 여전히 추정 계획입니다.

②·③·④는 Note 문구의 의미 경계를 뭉갰습니다. 카디널리티 피드백은 이전 수행의 실측을 근거로 다음 수행 계획을 다시 세우는 기능이지, 화면 계획에 단계별 실측 행 수(A-Rows)를 채워 넣는 기능이 아닙니다(②). 그 기능은 실제 수행이 있어야 발동하므로 문장을 수행하지 않는 **EXPLAIN PLAN**에는 나타날 수 없고(③), 동적 샘플링은 파싱 시점에 블록을 표본 조사할 뿐 문장을 끝까지 수행하는 것이 아닙니다(④).

| 선택지 | 판정 | 이유                                                                                          |
|-----|----|---------------------------------------------------------------------------------------------|
| ①   | ○  | 동적 샘플링 Note는 파싱 시점에 통계 부족 객체를 표본 조사해 카디널리티를 보정했다는 표시일 뿐, 계획은 수행 없이 얻은 추정 계획 그대로입니다          |
| ②   | ×  | 카디널리티 피드백은 이전 수행의 실측으로 다음 계획을 재최적화하는 기능이지, 화면 계획에 단계별 실측 행 수를 채워 실제 실행계획으로 바꾸는 것이 아닙니다      |
| ③   | ×  | 카디널리티 피드백은 실제 수행이 있어야 발동하므로, 문장을 수행하지 않는 <b>EXPLAIN PLAN+DISPLAY</b> 조합에는 그 Note가 나타나지 않습니다 |
| ④   | ×  | 동적 샘플링은 파싱 시점에 블록 일부를 표본 조사할 뿐 문장을 끝까지 수행하지 않으므로, 이 Note가 있어도 계획은 여전히 추정 계획입니다              |

#### ■ 이 문제의 핵심

1. 동적 샘플링은 파싱 시점에 통계 부족 객체를 표본 조사해 카디널리티를 보정하는 것으로, 계획은 여전히 추정입니다.
2. 카디널리티 피드백은 이전 수행의 실측을 근거로 다음 수행 계획을 재최적화하는 기능입니다.
3. 카디널리티 피드백은 실제 수행이 선행되어야 발동하므로, 수행 없는 **EXPLAIN PLAN**에는 나타나지 않습니다.
4. 두 Note 어느 쪽도 화면 계획에 단계별 실측 행 수(A-Rows)를 채워 실제 계획으로 바꾸지 않습니다.

■ 한 줄 정리 동적 샘플링은 파싱 시점의 표본 조사(계획은 추정 유지), 카디널리티 피드백은 수행 뒤의 재최적화 근거일 뿐, 둘 다 계획을 실측 계획으로 바꾸지 않으므로 동적 샘플링 Note를 "추정 보정"으로 정확히 읽은 ①만 맞습니다.

### 문제 5. 정답 ②

먼저 BCHR로 논리 대비 물리 읽기를 봅니다.

$$\begin{aligned} \text{BCHR} &= ((\text{Query} + \text{Current}) - \text{Disk}) / (\text{Query} + \text{Current}) \times 100 \\ &= (3,000,000 + 0 - 240,000) / 3,000,000 \times 100 \\ &= 2,760,000 / 3,000,000 \times 100 = 92.0\% \end{aligned}$$

$$\text{디스크 물리 읽기 비중} = 240,000 / 3,000,000 \times 100 = 8.0\%$$

캐시 적중은 92%로 나쁘지 않습니다. 다음은 시간의 정체입니다.

$$\text{CPU 비중} = 8.000 / 45.000 \times 100 \approx 17.8\% \rightarrow \text{나머지 82\%가 대기}$$

$$\text{Elapsed} - \text{CPU} = 45.000 - 8.000 = 37.0\text{초} \leftarrow \text{기다린 시간}$$

응답의 82%가 대기입니다. 대기가 난 곳을 Row Source로 찾되, cr(논리)과 pr·pw(물리·temp)를 분리해서 봅니다.

$$\text{테이블 스캔 물리 읽기} = \text{구독료내역 } 84,000 + \text{구독자 } 6,000 + \text{콘텐츠 } 0 = 90,000$$

$$\text{HASH GROUP BY pr} = 240,000, \text{ pw} = 150,000$$

→ 스캔 물리 90,000을 빼면 240,000 - 90,000 = 150,000이 spill 재읽기

$$\text{물리 읽기 } 240,000 \text{ 중 spill 비중} = 150,000 / 240,000 \times 100 = 62.5\%$$

**pw=150,000**은 **HASH GROUP BY** 노드에만 붙어 있습니다. 조인 노드는 pw=0이라 spill이 없고, 집계가 400만 그룹을 해시 영역에 다 담지 못해 temp로 흘린 것입니다. 그 150,000 블록을 되읽느라 물리 읽기의 62.5%가 났습니다. 시간도 이 노드에 몰립니다.

$$\text{구독료내역 Full Scan time} = 8.0\text{초} \leftarrow 1.5\text{억 건이어도 멀티블록이라 빠름}$$

$$\text{HASH GROUP BY time} = 44.0\text{초} \leftarrow \text{spill I/O로 정체된 지점}$$

cr은 구독료내역 Full Scan(2,900,000, 96.7%)이 지배하지만 그 스캔은 8초에 끝났습니다. 45초를 만든 것은 논리 읽기가 아니라 집계의 디스크 spill(pw=150,000)입니다. 따라서 처방은 PGA(해시/정렬 영역) 확대이며, ②가 BCHR과 병목 지목을 함께 옳게 했습니다.

| 선택지 | 판정 | 이유                                                                                                                                                                                                                                      |
|-----|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | Fetch의 CPU 8.000초·Elapsed 45.000초·cr 3,000,000은 그대로이지만, CPU는 Elapsed의 17.8%뿐이라 CPU 바운드가 아닙니다. 나머지 82%(37초)는 pw=150,000의 spill I/O 대기이므로, DOP 상향은 대기의 원인을 CPU로 오귀속한 처방입니다                                                                |
| ②   | ○  | BCHR (3,000,000-240,000)/3,000,000 = 92.0%가 정확하고, pw=150,000이 <b>HASH GROUP BY</b> 에만 붙어 물리 읽기의 62.5%가 400만 그룹의 집계 spill임을 짚어 PGA를 처방한 것이 맞습니다                                                                                          |
| ③   | ×  | cr 2,900,000(96.7%)·pr 84,000이 구독료내역 Full Scan인 것은 사실이나 그 노드의 time은 8.0초로 빠릅니다. 45초 정체는 논리 읽기가 아니라 pw=150,000의 집계 spill이므로, 인덱스로 cr을 줄이자는 것은 수치는 맞되 병목을 오귀속한 처방입니다                                                                      |
| ④   | ×  | 안쪽 <b>HASH JOIN</b> 의 cr=2,996,000·pr=90,000은 맞지만 그 노드의 pw는 0이라 조인은 spill하지 않았고, pr 90,000도 자식 스캔(84,000+6,000)의 누적일 뿐입니다. spill(pw=150,000)은 <b>HASH GROUP BY</b> 에서 났으므로 조인 순서를 바꿔도 400만 그룹의 집계 spill은 그대로여서, 원인을 조인 build로 오귀속한 것입니다 |



▪ 이 문제의 핵심

1. BCHR =  $((\text{Query} + \text{Current}) - \text{Disk}) / (\text{Query} + \text{Current}) \times 100$ . 여기서는  $(3,000,000 - 240,000) / 3,000,000 = 92.0\%$ 로 캐시 적중은 양호합니다.
  2. cr(논리)과 pr·pw(물리·temp)를 분리해야 합니다. cr은 구독료내역 Full Scan(96.7%)이 지배해도, 시간을 만든 건 pw=150,000의 집계 spill입니다.
  3. **pw>0**이 어느 노드에 붙었는지가 핵심입니다. 조인 노드는 pw=0, **HASH GROUP BY**만 pw=150,000이라 spill은 집계에서 났습니다 — 물리 읽기 240,000의 62.5%가 spill 재읽기입니다.
  4. 그룹 수가 큰 집계(400만 그룹)가 해시/정렬 영역을 넘기면 spill합니다 — 병목은 집계이지 큰 테이블 스캔(time 8초)이 아닙니다.
  5. 처방은 PGA(해시/정렬 영역) 확대입니다. DOP·인덱스·조인 순서는 37초의 spill 대가를 거당하지 못합니다.
- 한 줄 정리 BCHR 92%까지 옳게 읽어도, pw=150,000이 조인이 아니라 **HASH GROUP BY**에 붙어 물리 읽기의 62.5%가 400만 그룹의 집계 spill이라는 사실을 놓치면 병목을 CPU·구독료내역 스캔·조인 순서로 오귀속하게 된다.

문제 6. 정답 ②

플랜의 두 노드에서 나온 Card 값을 나란히 놓습니다.

INDEX RANGE SCAN 점검\_IX Card 12,000 ← 인덱스에서 훑고 남은 건수  
TABLE ACCESS BY INDEX ROWID 점검내역 Card 3,600 ← 테이블까지 거친 뒤 남은 건수

**점검\_IX**는 (관할서코드, 점검일시) 순서입니다. WHERE에는 관할서코드 = 'S07'(등치)와 점검일시 범위가 있으므로, 선두 칼럼 등치 + 두 번째 칼럼 범위까지가 인덱스로 연속해서 훑을 수 있는 구간입니다. 그 결과가 INDEX RANGE SCAN의 12,000건입니다.

판정결과는 **점검\_IX**의 구성 칼럼이 아닙니다. 인덱스에 값이 없으니 인덱스 단계에서는 걸러낼 수 없고, ROWID로 테이블 블록을 읽어 온 TABLE ACCESS BY INDEX ROWID 단계에서 필터로 처리됩니다. 그래서 12,000건이 이 단계를 거치며 3,600건으로 줄어듭니다.

12,000 (인덱스 액세스: 관할서코드=, 점검일시 범위)  
└─ 테이블 필터(판정결과='부적합') 적용 → 3,600 (최종)

즉 ②가 이 구조를 정확히 서술합니다. 인덱스 반환 건수를 최종 건수로 본 ①, 판정결과까지 인덱스 액세스 조건으로 끌어올린 ③, 선두 칼럼 등치까지 필터로 밀어 넣은 ④는 모두 이 Card 흐름과 어긋납니다.

| 선택지 | 판정 | 이유                                                                                                                                                                                                                    |
|-----|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | Cost 29·Card 12,000은 인덱스 단계 값일 뿐이고, 그 위 TABLE ACCESS BY INDEX ROWID(Cost 412)의 Card는 3,600입니다. 이 단계가 칼럼만 채우는 것이 아니라 판정결과 필터로 12,000→3,600의 건수 감소를 일으키며, NESTED LOOPS로 넘어가는 것도 3,600건입니다                               |
| ②   | ○  | 관할서코드(등치)·점검일시(범위)는 인덱스 액세스 조건, 판정결과는 인덱스에 없어 TABLE ACCESS 단계 필터 — Card가 12,000→3,600으로 줄어드는 흐름과 일치합니다                                                                                                                |
| ③   | ×  | 판정결과는 <b>점검_IX</b> 구성 칼럼이 아니므로 인덱스 액세스 조건이 될 수 없습니다. 세 조건이 함께 스캔 범위를 정했다면 INDEX RANGE SCAN Card가 이미 3,600이었을 텐데 실제로는 12,000이고, Id 2의 Card 3,600은 조인 이전인 점검내역 테이블 필터의 결과입니다 — 테이블 필터를 인덱스 액세스 조건 집합에 끼워 넣은 하위항목 전체화입니다 |
| ④   | ×  | 두 번째 칼럼이 범위라는 성질을 선두 칼럼에까지 확대한 진술입니다. 선두 칼럼 등치는 그 자체로 액세스 조건이고, 범위는 그 다음 칼럼부터 적용됩니다. 등치까지 필터였다면 940만 건을 전수 스캔한 셈이라 Cost가 29에 그칠 수 없고 오퍼레이션도 INDEX RANGE SCAN이 아니었을 것입니다                                               |

▪ 이 문제의 핵심

1. 결합 인덱스는 선두 칼럼 등치 + 다음 칼럼 범위까지가 액세스 조건. 여기서는 관할서코드(=)·점검일시(범위)가 INDEX RANGE SCAN 구간을 만듭니다(Card 12,000).
2. 인덱스에 없는 칼럼은 테이블 필터. 판정결과는 ROWID로 테이블을 읽은 뒤 TABLE ACCESS BY INDEX ROWID 단계에서 걸러집니다.
3. 두 노드의 Card 차이(12,000 vs 3,600)가 곧 테이블 필터의 효과입니다. 인덱스 반환 건수 ≠ 최종 건수.
4. 테이블 필터 조건을 인덱스 액세스 조건 집합에 끼워 넣지 않습니다. 인덱스에 없는 판정결과는 스캔 범위를 정하는 데 관여하지 못합니다.

- 한 줄 정리 선두 칼럼 등치(관할서코드)와 다음 칼럼 범위(점검일시)는 인덱스 액세스 조건이 되고, 인덱스에 없는 판정결과는 테이블 액세스 단계 필터가 되어 Card가 12,000에서 3,600으로 줄어듭니다.

## 문제 7. 정답 ④

Row Source의 cr는 누적이므로, 테이블 액세스 노드의 271,500에서 자식 인덱스 1,500을 빼면 테이블 자체 블록 I/O가 나옵니다. 이를 ROWID 수로 나눠 클러스터링 팩터를 가늠합니다.

테이블 자체 블록 I/O =  $271,500 - 1,500 = 270,000$

액세스 ROWID(행) 수 = 300,000

CF 추정 =  $270,000 / 300,000 \approx 0.90$  블록/행

→ 행마다 거의 다른 블록 → 클러스터링 팩터 최악( $\approx$  행 수에 근접)

CF가 나쁘면 인덱스로 뽑은 각 행이 서로 다른 테이블 블록에 흩어져 있어, 테이블 Random 액세스가 한 행당 블록 하나씩을 단일 블록으로 읽습니다. 어디에 시간이 쏟렸는지 봅시다.

테이블 액세스 cr 비중 =  $270,000 / 271,500 \times 100 \approx 99.4\%$  ← 논리 읽기의 대부분

인덱스 스캔 cr 비중 =  $1,500 / 271,500 \times 100 \approx 0.6\%$  ← 극소

테이블 액세스 time = 29.4초 (인덱스 스캔 time은 0.42초)

이제 손익분기점입니다. 테이블은 약 40만 블록인데, 인덱스 경유로 읽는 테이블 블록이 270,000입니다.

인덱스 경유 테이블 블록 = 270,000

테이블 총 블록 =  $50,000,000 / 125 = 400,000$

비율 =  $270,000 / 400,000 = 67.5\%$

→ 테이블의 2/3 가까이를 '단일 블록 랜덤 읽기'라는 느린 경로로 훑음

→ 다중 블록 Full Scan 손익분기점을 이미 넘음

인덱스 스캔(cr 1,500·0.42초)은 문제가 아닙니다. 정체는 CF 최악으로 부풀어 오른 테이블 Random 액세스 270,000블록이며, 이는 필요한 컬럼을 인덱스에 더한 커버링 인덱스로 테이블 액세스 자체를 없애거나 Full Scan으로 전환할 때 사라집니다. 따라서 ④가 원인을 정확히 지목했습니다.

| 선택지 | 판정 | 이유                                                                                                                                                                            |
|-----|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | cr 1,500은 맞지만 그것은 전체 271,500의 0.6%이고, <b>상품평_N1</b> 노드의 time은 0.42초라 29.995초 중 1.4%뿐입니다. 리프가 조각났다는 근거도 트레이스에 없고, 리빌드해도 270,000블록의 테이블 Random 액세스는 그대로이므로 병목을 인덱스로 오귀속한 것입니다   |
| ②   | ×  | Fetch 501회와 회당 600행(=300,000/500)은 맞지만, CPU 4.000초와 Elapsed 29.995초의 26초 격차는 왕복이 아니라 상위 노드 time 29.4초, 곧 270,000블록의 테이블 액세스에서 났습니다. 배열 크기를 키워도 읽어야 할 테이블 블록 수는 줄지 않습니다        |
| ③   | ×  | Disk 60,000이 cr 271,500의 22.1%인 것은 맞으나, 그 6만을 전부 캐시 적중으로 돌려도 테이블 자체 논리 읽기 270,000블록(단일 블록 랜덤)은 그대로 남습니다. 병목은 물리 읽기가 아니라 CF가 만든 논리 블록 액세스 횟수이므로 버퍼 캐시로 오귀속한 것입니다               |
| ④   | ○  | 테이블 자체 cr 270,000(=271,500-1,500)이 ROWID 300,000과 거의 같아 CF $\approx$ 0.90으로 최악이고, 이 270,000이 cr의 99.4%·29.4초를 차지하며 40만 블록의 67.5%를 랜덤 읽기로 훑어 손익분기점을 넘었다는 지목과 커버링 인덱스 처방이 정확합니다 |

### ▪ 이 문제의 핵심

1. Row Source cr는 누적입니다. 테이블 자체 블록 I/O = 상위(271,500) - 자식 인덱스(1,500) = 270,000. 이걸 ROWID 수로 나눠 CF를 가늠합니다( $270,000/300,000 \approx 0.90$  블록/행).
2. CF가 최악이면 테이블 Random 액세스가 병목입니다. 270,000이 cr의 99.4%·time 29.4초를 차지하고, 인덱스 스캔(1,500·0.42초)은 무죄입니다.
3. 손익분기점: 인덱스 경유 테이블 블록 270,000이 총 40만 블록의 67.5%를 단일 블록 랜덤으로 훑으면 다중 블록 Full Scan보다 불리합니다.
4. 처방은 인덱스 리빌드·배열 크기·버퍼 캐시가 아니라 테이블 액세스 자체 제거(커버링 인덱스) 또는 Full Scan 전환입니다.
  - 한 줄 정리 테이블 자체 cr 270,000이 ROWID 300,000과 거의 같아 CF가 최악이고, 이 270,000블록의 랜덤 액세스가 cr의 99.4%·29.4초를 차지하며 손익분기점(67.5%)을 넘었으므로 처방은 인덱스·캐시가 아니라 커버링 인덱스로 테이블 액세스를 없애는 것이다.

## 문제 8. 정답 ①

클러스터형 인덱스(IOT)의 보조 인덱스가 힙 테이블의 보조 인덱스와 결정적으로 다른 지점은 행 식별자로 무엇을 저장하는가입니다.

힙 테이블의 보조 인덱스 : 행 식별자 = 물리적 RID/ROWID → RID 따라 1회에 행 도달  
 클러스터형 인덱스의 보조 인덱스 : 행 식별자 = 클러스터 키(PK) 값(물리적 위치 아님)  
 → 보조 인덱스에서 키 찾기 → 클러스터형 인덱스를 키로 재탐색  
 → 리프에 도달해야 행 획득(2단계)

클러스터형 인덱스는 리프가 곧 데이터라서 행이 재배치될 때 물리적 위치가 바뀝니다. 그래서 보조 인덱스는 변하지 않는 클러스터 키 값을 행 식별자로 저장하고, 조회 시 그 키로 클러스터형 인덱스를 다시 타고 내려가 행에 닿습니다. ①은 이를 정반대로 뒤집어 "보조 인덱스가 물리적 RID를 담아 한 번에 행에 도달한다"고 했지만, RID를 담아 한 번에 닿는 것은 힙 테이블의 보조 인덱스입니다. 클러스터형 인덱스의 보조 인덱스는 클러스터 키를 저장해 2단계로 접근합니다.

②·③·④는 옳습니다. 클러스터형 인덱스가 리프=데이터라 별도 힙 세그먼트 없이 Index Unique Scan으로 리프 도달만으로 행을 얻는다는 점(②), 보조 인덱스가 클러스터 키를 식별자로 저장해 2단계 접근이 든다는 점(③), IOT의 긴 행이 오버플로 세그먼트로 갈라져 그 컬럼 접근에 추가 이동이 든다는 점(④)이 모두 실제 구조에 부합합니다.

| 선택지 | 판정 | 이유                                                                                                           |
|-----|----|--------------------------------------------------------------------------------------------------------------|
| ①   | ×  | 클러스터형 인덱스의 보조 인덱스는 물리적 RID가 아니라 클러스터 키(PK)를 행 식별자로 저장합니다. RID로 한 번에 도달하는 것은 힙 테이블의 보조 인덱스로, 둘을 뒤바꾼 정반대 서술입니다 |
| ②   | ○  | 클러스터형 인덱스는 리프가 곧 데이터라 별도 힙 세그먼트가 없고, 클러스터 키 Index Unique Scan은 리프 도달만으로 행을 얻어 테이블 액세스가 없습니다                  |
| ③   | ○  | 보조 인덱스는 클러스터 키를 식별자로 저장하므로, 보조 인덱스에서 키를 찾은 뒤 클러스터형 인덱스를 그 키로 재탐색하는 2단계 접근이 일어납니다                             |
| ④   | ○  | IOT는 임계 이상(또는 INCLUDING 지정) 컬럼을 오버플로 세그먼트에 갈라 저장하며, 그 컬럼을 읽을 때 리프에서 오버플로로 한 번 더 이동해야 합니다                     |

#### ■ 이 문제의 핵심

- 클러스터형 인덱스(IOT)는 리프가 곧 데이터여서 별도 힙 세그먼트가 없고, 키 Unique Scan은 리프 도달만으로 행을 얻습니다.
- 보조 인덱스의 행 식별자는 물리적 RID가 아니라 클러스터 키(PK) 값입니다.
- 그래서 보조 인덱스 조회는 보조 인덱스 → 클러스터형 인덱스 재탐색의 2단계로 행에 닿습니다.
- 오버플로 세그먼트로 갈라진 긴 행의 컬럼은 리프에서 한 번 더 이동해야 읽습니다.

■ 한 줄 정리 클러스터형 인덱스(IOT)의 보조 인덱스는 물리적 RID가 아니라 클러스터 키를 저장해 2단계로 행에 닿는데, "물리적 RID를 담아 한 번에 도달한다"는 ①은 힙 테이블의 동작을 끌어와 뒤바꾼 정반대 서술입니다.

### 문제 9. 정답 ④

인덱스 병합이 가능하다는 것과, 그것이 결합 인덱스를 온전히 대신한다는 것은 전혀 다른 이야기입니다. ④는 앞의 부분적 사실을 뒤의 전면적 결론으로 부풀렸습니다.

결합 인덱스(두 컬럼 묶음)

선두 컬럼 조건으로 스캔 구간을 한 번에 좁힘 → 읽는 리프가 적음

단일 컬럼 인덱스 두 개 + 병합(index merge)

인덱스 A의 조건 범위를 훑고, 인덱스 B의 조건 범위를 훑은 뒤

두 결과의 교집합을 다시 구함 → 두 인덱스를 각각 넓게 스캔 + 결합 비용

→ 특정 상황에서 쓰일 뿐, 결합 인덱스의 '선두 컬럼으로 구간 좁히기'를 재현하지 못함

병합은 두 인덱스를 각각 조건 범위만큼 훑은 뒤 교집합을 구하는 방식이라, 결합 인덱스가 선두 컬럼 하나로 스캔 구간을 곧장 좁히는 효율을 그대로 내지 못합니다. 특정 조건 분포에서 옵티마이저가 선택할 수 있는 하나의 경로일 뿐인데, ④는 이를 "두 컬럼을 함께 거는 조회에서 결합 인덱스를 온전히 대신한다"는 전면적 결론으로 확대해 "결합 인덱스가 필요 없다"고 단정했습니다. 부분적으로만 성립하는 이점을 전체 등식으로 끼워 넣은 서술입니다.

①·②·③은 옳습니다. 두 컬럼 등가 조건에는 선택도 높은 컬럼을 선두로 둔 결합 인덱스가 유리하다는 점(①), 등가는 앞·범위는 뒤라는 컬럼 순서 원칙(②), 결합 인덱스로 묶어 인덱스 수를 줄이면 DML 부하 관리에 유리하다는 점(③)이 모두 인덱스 설계의 실제 기준에 부합합니다.

| 선택지 | 판정 | 이유                                                                                                                    |
|-----|----|-----------------------------------------------------------------------------------------------------------------------|
| ①   | ○  | 두 컬럼 등가 조건에는 선택도 높은 컬럼을 선두로 둔 결합 인덱스가 스캔 구간을 좁혀, 단일 컬럼 인덱스 둘을 두는 것보다 대체로 효율적입니다                                       |
| ②   | ○  | 컬럼 순서는 조희 패턴을 따라 등가 조건을 앞, 범위 조건을 뒤에 두어야 선행 컬럼에서 구간을 좁힌 뒤 후행 범위를 이어 갈 수 있습니다                                          |
| ③   | ○  | 컬럼 수가 늘면 리프 엔트리가 길어지고 DML 유지 부하가 커지므로, 결합 인덱스로 묶어 인덱스 개수를 줄이는 편이 DML 부하 관리에 유리한 경우가 많습니다                              |
| ④   | ×  | 인덱스 병합은 두 인덱스를 각각 훑어 교집합을 구하는 특정 경로일 뿐, 결합 인덱스의 선두 컬럼 구간 좁히기를 재현하지 못합니다. "온전히 대신하니 결합 인덱스가 불필요"는 부분 이점을 전체로 부풀린 서술입니다 |

#### ■ 이 문제의 핵심

1. 두 컬럼 등가 조건에는 선택도 높은 컬럼을 선두로 둔 결합 인덱스가 대체로 유리합니다.
2. 컬럼 순서는 조희 패턴을 따라 등가는 앞, 범위는 뒤에 둡니다.
3. 결합 인덱스로 묶어 개수를 줄이면 DML 유지 부하 관리에 유리합니다.
4. 인덱스 병합은 특정 경로로 가능할 뿐, 결합 인덱스의 선두 컬럼 구간 좁히기를 온전히 대신하지 못합니다.

■ 한 줄 정리 인덱스 병합은 두 인덱스를 각각 훑어 교집합을 구하는 특정 경로일 뿐이라 결합 인덱스의 선두 컬럼 구간 좁히기를 재현하지 못하는데, 이를 "결합 인덱스를 온전히 대신하니 불필요"로 부풀린 ④는 부분 이점을 전체 등식으로 확대한 서술입니다.

### 문제 10. 정답 ②

최상단 Id 1의 오퍼레이션 이름을 그대로 읽습니다.

Id 1 FILTER ← 서브쿼리가 Unnesting되지 않고 필터로 남은 노드  
 Id 2 TABLE ACCESS FULL 점종대상 (구동 집합: 대상자 각 1건)  
 Id 3~4 점종기록 (INDEX RANGE SCAN 점종일자 범위) ← 각 구동 행마다 확인

**FILTER** 오퍼레이션이 최상단에 있다는 것은, 옵티마이저가 **EXISTS** 서브쿼리를 세미 조인으로 Unnesting하지 않았다는 뜻입니다. 서브쿼리는 상관 서브쿼리 그대로 남아, 구동 테이블 점종대상(Id 2)의 한 행을 읽을 때마다 서브쿼리(Id 3~4)를 다시 수행합니다. 대상자ID 상관 변수를 서브쿼리에 넣고 점종기록을 인덱스로 뒤져, 조건을 만족하는 행이 하나라도 나오면 그 대상자를 통과시킵니다.

다만 오라클 **FILTER**는 입력값 캐싱(subquery caching)을 합니다. 이미 서브쿼리를 수행해 본 대상자ID 값이 다시 들어오면 저장된 결과를 재사용해 재수행을 건너뛸니다. 그래서 "구동 행마다 반복 수행 + 중복 값은 캐싱으로 회피"가 **FILTER**의 정확한 특성입니다. 이를 서술한 것이 ②입니다.

FILTER (Unnesting 안 됨) : 구동 행마다 서브쿼리 반복 수행 + 입력값 캐싱 (② 서술)  
 HASH JOIN SEMI (Unnesting됨) : 조인 1회로 매칭, 반복 수행 없음 (이 플랜 아님)  
 HASH JOIN ANTI : NOT EXISTS 변환, 매칭 없는 행만 통과 (이 플랜 아님)

Id 1이 **HASH JOIN SEMI**였다면 서브쿼리를 조인 한 번으로 풀어 반복이 없었겠지만(①), 이 플랜은 **FILTER**입니다. **HASH JOIN ANTI**로 본 ③, **FILTER**를 세미 조인 동작(점종기록 전체를 미리 해시 적재)으로 뒤바꾼 ④도 노드 이름·동작과 어긋납니다.

| 선택지 | 판정 | 이유                                                                                                                                                                                                                        |
|-----|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | 상관 조건이 대상자ID 등치인 것은 맞지만, 등치라고 항상 Unnesting되는 것은 아닙니다. Id 1은 <b>HASH JOIN SEMI</b> 가 아니라 <b>FILTER</b> 이고, Id 3~4가 그 아래에 자식으로 붙어 있어 점종대상 120만 건의 구동 행마다 반복 수행됩니다 — 세미 조인과 <b>FILTER</b> 를 뒤바꾼 진술입니다                        |
| ②   | ○  | Id 1 <b>FILTER</b> 는 서브쿼리가 Unnesting되지 않은 노드로, 점종대상(Id 2) 각 행마다 서브쿼리(Id 3~4)를 반복 수행하되 이미 처리한 대상자ID는 입력값 캐싱으로 재수행을 건너뛸니다. <b>FILTER</b> 의 정확한 특성입니다                                                                        |
| ③   | ×  | Id 1은 <b>FILTER</b> 이지 <b>HASH JOIN ANTI</b> 가 아니고, SQL도 <b>NOT EXISTS</b> 가 아니라 <b>EXISTS</b> 입니다. 점종일자 2026-06-01 이후 행이 하나라도 잡히는 대상자를 걸러 내는 것이 아니라 반대로 통과시키므로, 결과 집합을 정확히 뒤집어 놓은 진술입니다                                  |
| ④   | ×  | Id 4 <b>INDEX RANGE SCAN 점종기록_IX</b> 는 Id 1 <b>FILTER</b> 의 자식이라 구동 행의 대상자ID를 받아 그때그때 수행되지, 4,100만 건에서 대상자ID를 미리 뽑아 해시로 적재하지 않습니다. 미리 적재해 조인 한 번으로 끝내는 것은 세미 조인(Unnesting) 동작이므로, <b>FILTER</b> 에 세미 조인의 동작을 갖다 붙인 값스왑입니다 |

#### ■ 이 문제의 핵심

1. 최상단 **FILTER**는 서브쿼리 Unnesting이 안 된 신호입니다. 서브쿼리가 상관 서브쿼리 그대로 남아 필터로 수행됩니다.

2. FILTER는 구동 행마다 서브쿼리를 반복 수행합니다. 접종대상 한 행을 읽을 때마다 접종기록 서브쿼리를 다시 돌립니다.
3. 오라클 FILTER는 입력값 캐싱을 합니다. 이미 확인한 상관 변수(대상자ID) 값은 캐싱된 결과를 재사용해 재수행을 건너뛰니다.
4. FILTER ≠ HASH JOIN SEMI ≠ HASH JOIN ANTI. 세미는 Unnesting되어 조인 1회, 안티는 **NOT EXISTS**의 매칭 없는 행만 통과 — 이 플랜은 그중 어느 것도 아닌 **FILTER**입니다.
  - 한 줄 정리 최상단 Id 1이 **FILTER**이므로 서브쿼리는 Unnesting되지 않은 채 접종대상 각 행마다 반복 수행되며 중복 대상자ID는 입력값 캐싱으로 회피하므로, 이를 세미 조인(①)·안티 조인(③)·세미 조인 동작(④)으로 본 진술은 모두 틀립니다.

## 문제 11. 정답 ①

소트 머지 조인의 비용은 크게 양쪽을 조인 키로 정렬하는 비용 + 정렬된 두 스트림을 병합하는 비용으로 나뉩니다. 이 중 정렬 비용이 사라지면 소트 머지 조인은 매우 저렴해집니다.

일반적 소트 머지 조인 : [양쪽 Sort Join] → [Merge] ← 정렬 비용이 큼  
 인덱스로 정렬 확보 시 : [정렬 생략] → [Merge] ← 병합 비용만 남음  
 해시 조인 : [Build Input로 해시 테이블 생성] → [Probe]

조인 키에 인덱스가 있어 양쪽을 이미 정렬된 순서로 읽어 올 수 있으면, 소트 머지 조인은 부담이 큰 정렬 단계를 건너뛰고 병합만 하면 됩니다. 반면 해시 조인은 이 경우에도 여전히 한쪽으로 해시 테이블을 만들어야 하므로, 정렬이 생략된 소트 머지 조인이 상대적으로 유리해질 수 있습니다. ①이 처리 구조와 선택 기준을 정확히 짚습니다.

나머지는 키워드에서 곧장 결론으로 건너뛴 반사적 연상입니다.

②: 해시 조인이 버킷 단위 등치 매칭이라 비등치를 못 푼다는 앞부분은 맞지만, '비등치=소트 머지 전용'은 반사입니다. NL 조인은 드라이빙 행마다 상대 집합을 훑어 조건을 평가하므로 비등치 조인도 처리합니다(인덱스 활용이 어려워 뿐입니다).

③: Sort Join이 조인 키로 정렬한다는 앞부분은 맞지만, '정렬한다=모든 정렬이 공짜'는 반사입니다. 소트 머지 조인이 순서를 보장하는 것은 조인 키 기준뿐이며, 조인 키가 아닌 컬럼의 **ORDER BY**는 재사용할 순서가 없어 별도 정렬이 필요합니다.

④: 해시 조인이 Build Input으로 work area를 소비한다는 앞부분은 맞지만, '소트=메모리 안 씬'은 반사입니다. Sort Join은 스트리밍이 아니라 양쪽을 통째로 정렬하는 단계라 PGA의 work area(Sort Area)를 쓰고 부족하면 Temp로 넘칩니다.

| 선택지 | 판정 | 이유                                                                                                                                                           |
|-----|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ○  | 인덱스로 양쪽을 정렬된 순서로 읽으면 정렬 단계가 생략돼 병합만 남으므로, 해시 테이블을 새로 만드는 해시 조인보다 유리해질 수 있습니다                                                                                 |
| ②   | ×  | 해시 조인이 버킷 단위 등치 매칭이라 부등호·범위를 못 푼다는 앞부분은 맞지만 결론이 틀렸습니다. NL 조인도 드라이빙 행마다 상대를 훑어 비등치 조건을 평가하므로 비등치 조인을 수행합니다 — 인덱스 활용이 어려워 비효율적일 뿐이며, '비등치=소트 머지 전용'은 반사적 연상입니다 |
| ③   | ×  | Sort Join이 양쪽을 조인 키로 정렬한다는 앞부분은 맞지만, 그 스트림이 보장하는 순서는 조인 키 기준뿐입니다. 조인 키가 아닌 컬럼의 <b>ORDER BY</b> 는 재사용할 순서가 없어 별도 SORT ORDER BY가 뒤에 붙습니다                       |
| ④   | ×  | 해시 조인이 Build Input으로 work area를 크게 쓴다는 앞부분은 맞지만, Sort Join은 스트리밍이 아니라 양쪽을 통째로 정렬하는 단계라 같은 PGA work area에서 수행되고 부족하면 Temp로 넘칩니다. 메모리 절약 수단이라는 결론이 틀렸습니다       |

### ▪ 이 문제의 핵심

1. 정렬이 생략되면 소트 머지 조인이 저렴해진다. 인덱스로 양쪽을 정렬된 순서로 읽으면 병합 비용만 남아 해시 조인의 대안이 됩니다.
2. 비등치 조인은 NL 조인도 처리한다. 소트 머지 조인 전용이 아니며, NL은 드라이빙 건마다 상대를 훑어 조건을 평가합니다.
3. 정렬이 보장하는 순서는 조인 키 기준뿐이다. 조인 키가 아닌 컬럼의 정렬까지 공짜로 해결되지는 않습니다.
4. 정렬도 메모리를 쓴다. 정렬은 PGA work area에서 이뤄지고 부족하면 Temp로 넘치므로, 메모리 절약 수단으로 볼 수 없습니다.

▪ 한 줄 정리 인덱스로 양쪽을 정렬된 순서로 읽어 정렬을 생략할 수 있으면 소트 머지 조인은 병합만 남아 해시 조인의 대안이 되며, "비등치=소트 머지 전용"·"정렬하니 모든 ORDER BY가 공짜"·"소트는 메모리를 안 씬"은 키워드에서 곧장 결론으로 건너뛴 반사적 연상입니다.

## 문제 12. 정답 ①

복합 뷰 Merging은 비용 기반(cost-based) 변환입니다. 이름·인상에서 "합치면 선택 폭이 넓어지니 무조건 이득"이라고 건너뛰면 틀립니다.

단순 뷰 병합 : 결과가 항상 동일 → 규칙적으로(휴리스틱) 병합  
 복합 뷰 Merging : GROUP BY·DISTINCT 등 포함 → 병합 후 비용을 건져 채택 여부 결정  
 · 병합이 유리 → 병합  
 · 병합이 불리 → 병합하지 않고 뷰를 개별 수행

①은 "복합 뷰 Merging이 선택 폭을 넓힌다"는 표면 인상에서 "그러니 가능하지만 하면 비용 비교 없이 병합한다"는 결론을 반사적으로 끌어냈습니다. 그러나 집계·DISTINCT를 담은 복합 뷰의 병합은 병합했을 때가 오히려 비쌀 수 있어, 옵티마이저는 병합 전후의 비용을 비교해 채택 여부를 정합니다. '문법적으로 가능=무조건 병합'이 아니므로 ①이 부적절합니다.

나머지는 모두 옳습니다.

②: 복합 뷰 병합은 비용 기반이라, 불리하다고 판단되면 병합하지 않고 뷰를 개별 수행합니다.

③: 병합이 막혀 뷰가 독립 블록으로 남아도 조건절 Pushing으로 바깥 조건을 뷰 안에 밀어 넣어 처리 행을 줄일 수 있습니다.

④: 서브쿼리 Unnesting이 성사되면 그 조인 블록에 조인 순서·방식 선택지가 새로 생기는 등, 한 변환의 성사가 뒤이은 최적화의 여지를 넓히기도 합니다.

| 선택지 | 판정 | 이유                                                                                 |
|-----|----|------------------------------------------------------------------------------------|
| ①   | ×  | 복합 뷰 Merging은 비용 기반 변환이라 문법적으로 가능해도 병합이 불리하면 하지 않습니다. '선택 폭을 넓힘=무조건 병합'은 반사적 연상입니다 |
| ②   | ○  | GROUP BY·DISTINCT를 담은 복합 뷰 병합은 단순 병합과 달리 비용으로 판단되어, 불리하면 뷰를 개별 수행합니다               |
| ③   | ○  | 병합이 막혀 독립 블록으로 남아도 조건절 Pushing으로 바깥 조건을 뷰 안에 밀어 넣어 처리 행을 줄일 수 있습니다                 |
| ④   | ○  | Unnesting이 성사되면 그 조인 블록에 순서·방식 선택지가 생기는 등, 한 변환의 성사가 뒤이은 최적화의 여지를 넓히기도 합니다         |

### ■ 이 문제의 핵심

- 복합 뷰 Merging은 비용 기반 변환. 문법적으로 가능해도 병합이 불리하면 옵티마이저는 병합하지 않습니다.
- 단순 병합과 복합 병합은 판단 방식이 다르다. 단순 뷰는 규칙적으로, GROUP BY·DISTINCT를 담은 복합 뷰는 비용으로 판단합니다.
- 병합이 막혀도 조건절 Pushing이 대안. 바깥 조건을 뷰 안으로 밀어 넣어 처리 행을 줄일 수 있습니다.
- 변환은 서로 여지를 넓히기도 한다. 한 변환의 성사가 뒤이은 조인 순서·방식 선택 같은 최적화 기회를 여는 상호작용이 있습니다.

■ 한 줄 정리 복합 뷰 Merging은 병합 전후 비용을 건져 채택하는 비용 기반 변환이므로, "선택 폭을 넓히니 문법적으로 가능하면 비용 비교 없이 병합한다"는 진술은 표면 인상에서 결론을 끌어낸 반사적 연상입니다.

## 문제 13. 정답 ③

커서 공유의 기준은 SQL 텍스트가 문자 단위로 같은가입니다. 기본값 CURSOR\_SHARING = EXACT는 두 SQL이 부모 커서를 공유하려면 텍스트가 바이트 단위로 완전히 같아야 한다고 요구합니다 — 공백·대소문자·주석·리터럴 어느 하나만 달라도 서로 다른 SQL\_ID로 갈립니다.

③은 "EXACT가 공백·대소문자·주석을 정규화해 비교한다"는 정규화 가정의 오류입니다. EXACT는 텍스트를 정규화하지 않습니다. WHERE 상태='OPEN'과 where 상태 = 'OPEN'은 리터럴이 같아도 대소문자·공백이 달라 다른 커서로 하드파싱됩니다. 텍스트 차이를 흡수해 바인드처럼 묶어 주는 동작은 EXACT가 아니라 CURSOR\_SHARING = FORCE(리터럴을 시스템 바인드로 치환)에서만 일어나며, 그마저도 리터럴만 치환할 뿐 공백·대소문자·주석 차이는 여전히 서로 다른 텍스트로 남습니다. 그래서 부적절한 진술은 ③입니다.

①②④는 각각 EXACT에서 리터럴별 커서가 쌓여 mutex 경합이 커진다는 점(①), FORCE가 리터럴을 :SYS\_B\_n으로 치환해 커서를 공유시킨다는 점(②), FORCE가 바인드 피킹된 계획 하나를 공유해 편중 분포에서 계획 품질이 흔들릴 수 있다는 점(④)을 옳게 서술합니다.



| 선택지 | 판정 | 이유                                                                                         |
|-----|----|--------------------------------------------------------------------------------------------|
| ①   | ○  | 리터럴만 다른 30가지 SQL이 각각 하드파싱되어 라이브러리 캐시에 커서가 쌓이고, 파싱 경로의 <b>library cache: mutex</b> 경합이 커집니다 |
| ②   | ○  | FORCE는 리터럴을 <b>:SYS_B_0</b> 같은 시스템 바인드로 치환해 30가지 텍스트를 하나로 고정하므로, 커서를 공유하고 하드파싱이 줄어듭니다      |
| ③   | ×  | EXACT는 텍스트를 정규화하지 않습니다. 공백·대소문자·주석이 하나만 달라도 다른 SQL_ID이며, 리터럴 자동 치환은 FORCE에서만 일어납니다         |
| ④   | ○  | FORCE는 최초 실행의 바인드 피킹 계획 하나를 공유하므로, 값별 최적 계획이 다른 편중 컬럼에서는 불리한 계획이 재사용될 수 있습니다               |

#### ■ 이 문제의 핵심

- 기본(EXACT) 커서 공유 기준은 텍스트 완전 일치입니다. 공백·대소문자·주석·리터럴 어느 하나만 달라도 다른 커서로 하드파싱됩니다 — EXACT는 텍스트를 정규화하지 않습니다.
- 리터럴 결합 SQL은 값 조합만큼 커서가 걸립니다. 30가지 조합이면 30개 커서가 쌓여 파싱 부하와 **library cache: mutex** 경합을 부릅니다.
- CURSOR\_SHARING = FORCE**는 리터럴을 시스템 바인드로 치환해 텍스트를 하나로 만들어 커서를 공유시킵니다. 다만 공백·대소문자·주석 차이까지 흡수하지는 않습니다.
- FORCE의 이면은 바인드 피킹 계획 공유입니다. 편중 분포 컬럼에서는 값별 최적 계획이 아니라 하나의 공유 계획이 재사용돼 성능이 흔들릴 수 있습니다.

■ 한 줄 정리 기본(EXACT) 모드는 텍스트를 정규화하지 않아 공백·대소문자·주석·리터럴이 완전히 같아야만 커서를 공유하므로, "EXACT가 공백·대소문자를 정규화해 리터럴만 같으면 묶어 준다"는 ③은 착각입니다.

### 문제 14. 정답 ②

통계 수집에는 서로 독립인 여러 축이 있습니다. ②는 그중 '샘플 크기' 축만을 정확히 서술합니다.

[축 A] 샘플 크기 : 전수(100%) ↔ 표본(ESTIMATE\_PERCENT / AUTO\_SAMPLE\_SIZE) ← 표본 오차 관련  
 [축 B] 최신성(stale) : 수집 후 데이터 변경량이 임계치를 넘으면 stale ← 시간·변경량 관련  
 [축 C] 커서 재최적화 : 통계 갱신 → 커서 무효화 → '다음 파싱 때' 새 계획 생성 ← 파싱 시점 관련  
 [축 D] 동적 샘플링 : 파싱 시점에 만드는 일회성 즉석 통계(저장 통계와 별개)

**AUTO\_SAMPLE\_SIZE**는 옵티마이저가 객체 특성에 맞춰 샘플 크기를 자동 결정해, 전수 조사보다 훨씬 적은 비용으로도 실용적 정확도의 통계를 얻게 합니다(축 A). 이것은 최신성·재최적화·동적 샘플링과 얽히지 않는 독립적 사실이라 ②가 옳습니다.

나머지는 서로 다른 축을 하나로 묶은 별개 축 혼동입니다.

- ①: 동적 샘플링(축 D)은 파싱 시점의 일회성 통계일 뿐, **DBMS\_STATS**로 저장해 둔 통계의 stale 상태(축 B)를 갱신하지 않습니다.
- ③: 샘플 크기(축 A)와 최신성(축 B)은 다른 축입니다. 100% 수집으로 표본 오차를 없애도, 이후 데이터가 대량 변경되면 통계는 stale이 됩니다.
- ④: 통계 갱신(축 B)과 커서 재최적화(축 C)는 다른 축입니다. 통계 갱신은 커서를 무효화할 뿐, 새 계획은 다음 파싱 때 만들어지지 계획이 그 즉시 교체되는 것이 아닙니다.

| 선택지 | 판정 | 이유                                                                               |
|-----|----|----------------------------------------------------------------------------------|
| ①   | ×  | 동적 샘플링은 파싱 시점의 일회성 즉석 통계로, 저장된 통계의 stale 상태를 해소하지 않습니다. 두 축을 인과로 엮은 혼동입니다        |
| ②   | ○  | <b>AUTO_SAMPLE_SIZE</b> 는 옵티마이저가 샘플 크기를 자동 결정해, 전수보다 적은 비용으로 실용적 정확도의 통계를 얻게 합니다 |
| ③   | ×  | 표본 오차(샘플 크기)와 최신성(stale)은 다른 축입니다. 100% 수집이라도 이후 대량 변경되면 통계는 stale이 됩니다          |
| ④   | ×  | 통계 갱신은 커서를 무효화할 뿐, 새 실행계획은 다음 파싱 때 만들어집니다. 갱신 순간 계획이 즉시 교체되는 것이 아닙니다             |

#### ■ 이 문제의 핵심

1. **AUTO\_SAMPLE\_SIZE**는 샘플 크기를 자동 결정한다. 전수 조사보다 적은 비용으로 실용적 정확도의 통계를 얻습니다.



2. 동적 샘플링 ≠ 저장 통계 갱신. 동적 샘플링은 파싱 시점의 일회성 통계로, **DBMS\_STATS** 통계의 stale을 해소하지 않습니다.

3. 표본 오차와 최신성은 다른 축. 100% 수집이라도 이후 데이터가 대량 변경되면 통계는 stale이 됩니다.

4. 통계 갱신과 계획 교체는 다른 시점. 갱신은 커서를 무효화할 뿐, 새 계획은 다음 파싱 때 생성됩니다.

▪ 한 줄 정리 **AUTO\_SAMPLE\_SIZE**가 샘플 크기를 자동 결정한다는 진술만 옳으며, 동적 샘플링·전수 수집·자동 작업을 각각 저장 통계 갱신·최신성·계획 즉시 교체와 하나로 엮은 나머지는 독립인 축을 인과로 묶은 별개 축 혼동입니다.

### 문제 15. 정답 ③

MERGE의 **WHEN MATCHED ... DELETE WHERE**는 독립적인 삭제문이 아니라 UPDATE에 딸린 조건부 삭제입니다. 삭제 대상은 두 조건을 모두 만족하는 행뿐입니다.

DELETE WHERE의 삭제 대상 =

- (1) ON 조건으로 매칭되어 → WHEN MATCHED의 UPDATE가 이미 적용된 행 중
- (2) UPDATE로 '갱신된 뒤의 값'이 DELETE WHERE 조건을 만족하는 행

⇒ ON에 매칭되지 않은 t의 행은 애초에 UPDATE·DELETE 후보가 아니다

⇒ 판정 기준은 '원래 값'이 아니라 'UPDATE된 값'이다

③은 두 군데를 뒤집었습니다. 첫째, DELETE WHERE는 대상 테이블 전체가 아니라 ON으로 매칭돼 UPDATE된 행에만 적용됩니다 — 매칭되지 않은 행은 지워지지 않습니다. 둘째, 조건 평가 기준은 UPDATE 이전의 원래 값이 아니라 갱신된 뒤의 값입니다. ③은 "전체 테이블·원래 값·미매칭 행까지 삭제"라고 정반대로 서술했습니다. 그래서 부적절한 것은 ③입니다.

①은 MERGE의 매칭·분기·Upsert 동작을, ②는 DELETE WHERE가 매칭·갱신 행에 한정되고 갱신 후 값으로 평가된다는 점을, ④는 소스 중복이 안정적 행 집합을 깨 ORA-30926으로 실패하는 조건을 옳게 서술했습니다.

| 선택지 | 판정 | 이유                                                                                |
|-----|----|-----------------------------------------------------------------------------------|
| ①   | ○  | ON 매칭 시 UPDATE(+조건부 DELETE), 미매칭 소스는 INSERT를 한 문장으로 처리합니다. (나)의 구조 그대로입니다         |
| ②   | ○  | DELETE WHERE는 매칭돼 UPDATE된 행에만, 갱신 뒤 값 기준으로 적용됩니다. 미매칭 행은 이 절로 삭제되지 않습니다           |
| ③   | ×  | DELETE WHERE는 매칭·갱신된 행에만 걸리고 갱신 후 값으로 평가됩니다 — "전 테이블·원래 값·미매칭 행까지 삭제"는 양쪽 다 반대입니다 |
| ④   | ○  | 소스에 같은 키가 중복이면 대상 한 행에 두 소스가 매칭돼 갱신값이 모호해지므로, 안정적 행 집합을 얻지 못해 ORA-30926으로 실패합니다   |

#### ▪ 이 문제의 핵심

1. MERGE의 DELETE WHERE는 UPDATE에 딸린 조건부 삭제입니다. ON으로 매칭돼 UPDATE된 행만 삭제 후보이며, 매칭되지 않은 행은 지워지지 않습니다.

2. DELETE 조건은 갱신된 뒤의 값으로 평가합니다. UPDATE 이전 원래 값이 아니라 SET으로 바뀐 값을 기준으로 삭제 여부를 판정합니다.

3. 소스 중복은 ORA-30926을 부릅니다. 대상 한 행에 두 소스 행이 매칭되면 갱신값이 모호해져 안정적 행 집합을 얻지 못합니다.

4. MERGE는 수정 가능 조인 뷰 UPDATE의 대안입니다 — key-preserved 제약에 얽매이지 않고 조인 결과의 Upsert·조건부 삭제를 한 문장으로 처리합니다.

▪ 한 줄 정리 MERGE의 DELETE WHERE는 ON으로 매칭돼 UPDATE된 행에만, 갱신된 뒤의 값을 기준으로 걸리는데, ③은 "전 테이블을 원래 값 기준으로 훑어 미매칭 행까지 삭제"라 정반대로 서술했다 부적절합니다.

### 문제 16. 정답 ①

LIST 파티션의 Partition Pruning은 옵티마이저가 WHERE 조건의 값을 각 파티션의 값 목록(VALUES 리스트)과 대조해 스캔 대상을 좁히는 최적화입니다. 이 대조가 가능하려면 조건이 파티션 키 칼럼(시도코드) 자체에 걸려 있어야 합니다.

- ① 시도코드 = '27' → '27' ∈ p\_yeongnam → 1개 파티션 (pruning ○)
- ② 시도코드 IN ('11', '26') → '11' ∈ p\_capital, '26' ∈ p\_yeongnam → 2개 파티션 (전 스캔 아님)
- ③ SUBSTR(시도코드, 1, 2) = '41' → 키에 함수 → 값 목록 대조 불가 → 전 파티션 스캔
- ④ 시도코드 <> '11' → p\_capital엔 '28', '41'도 있어 제외 불가 → 전 파티션 스캔

①은 등치 조건이 파티션 키에 직접 걸려, '27'이 속한 p\_yeongnam 하나만 남기고 나머지를 건너뛸니다 — LIST pruning이 정확히 작동합니다. 그래서 옳은 설명은 ①입니다.

②는 틀렸습니다. LIST 파티션은 IN 리스트도 값 목록과 대조해 pruning하므로, 전 스캔이 아니라 p\_capital·p\_yeongnam 두 파티션만 읽습니다. ③은 부분적 진실(SUBSTR 결과가 원본과 같음)로 최선을 최악으로 뒤집는 형변환 함정입니다 — 결과값이 같아도 파티션 키에 함수를 씌우면 옵티마이저는 그 결과를 값 목록과 대조할 수 없어 pruning을 포기하고 전 파티션을 스캔합니다. ④는 <> 조건이 문제입니다. p\_capital은 '11' 말고 '28'·'41'도 담으므로 '11'만 배제한다고 p\_capital을 통째로 뺄 수 없어, 결국 모든 파티션을 스캔합니다.

| 선택지 | 판정 | 이유                                                                                     |
|-----|----|----------------------------------------------------------------------------------------|
| ①   | ○  | 등치 조건이 파티션 키에 직접 걸려 '27'이 속한 p_yeongnam 한 파티션만 접근하고 나머지는 건너뛸니다                         |
| ②   | ×  | LIST 파티션은 IN 리스트도 값 목록과 대조해 pruning합니다. 전 스캔이 아니라 p_capital·p_yeongnam 두 파티션만 읽습니다     |
| ③   | ×  | 결과값이 원본과 같아도 파티션 키에 SUBSTR을 씌우면 값 목록 대조가 불가능해집니다. pruning을 포기하고 전 파티션을 스캔합니다           |
| ④   | ×  | p_capital에는 '28'·'41'도 있어 '11'만 빼도 파티션을 통째로 제외할 수 없습니다. <>로는 pruning되지 않고 전 파티션을 스캔합니다 |

#### ▪ 이 문제의 핵심

1. LIST Partition Pruning은 파티션 키 값을 VALUES 목록과 대조해 작동합니다. 조건이 키 칼럼 자체에 직접 걸려야 합니다.
2. IN 리스트도 pruning 대상입니다. 각 값이 속한 파티션만 남기므로 여러 값이라도 전 스캔이 아니라 해당 파티션들만 읽습니다.
3. 키를 가공하면 pruning이 깨집니다. **SUBSTR(시도코드,...)**처럼 결과가 원본과 같아 보여도, 함수를 씌우면 값 목록 대조가 불가능해 전 파티션을 스캔합니다.
4. <>(부등호)는 LIST에서 pruning되기 어렵습니다. 한 파티션이 여러 값을 담으면 특정 값 하나를 배제해도 그 파티션 전체를 제외할 수 없습니다.

▪ 한 줄 정리 LIST pruning은 파티션 키 등치·IN에서 해당 파티션만 남기지만, **SUBSTR** 같은 키 가공은 결과가 같아 보여도 값 목록 대조를 막아 전 파티션을 스캔하므로, 정확히 pruning되는 것은 ①입니다.

### 문제 17. 정답 ②

분배 방식은 각 입력 위의 **PX SEND** 종류와 TQ 번호로 읽습니다. 이 플랜에는 재분배 지점이 세 곳입니다.

```

Id 9  PX SEND HASH :TQ10000  (Q1,00)  급수전 → 조인 키(급수전번호) 해시 재분배
Id 13 PX SEND HASH :TQ10001  (Q1,01)  급수사용량 → 조인 키 해시 재분배
Id 7  HASH JOIN      (Q1,02)          재분배된 두 입력을 hash-hash로 조인
Id 6  HASH GROUP BY (Q1,02)          조인 직후 부분(partial) 집계
Id 5  PX SEND HASH :TQ10002  (Q1,02)  그룹 키(지역코드) 해시 재분배 ← 조인과 별개
Id 3  HASH GROUP BY (Q1,03)          최종 집계
  
```

조인 재분배(hash-hash): 두 테이블은 급수전번호로 파티셔닝돼 있지 않으므로, 조인하려면 양쪽을 조인 키로 맞춰 재분배해야 합니다. Id 9·Id 13의 두 **PX SEND HASH**(TQ10000·TQ10001)가 급수전·급수사용량을 급수전번호 해시로 각각 재분배하고, Id 7 **HASH JOIN**(Q1,02)이 같은 버킷끼리 조인합니다.

집계 재분배는 조인과 별개: 집계 키(지역코드)는 조인 키(급수전번호)와 다릅니다. 그래서 조인 결과를 바로 최종 집계할 수 없고, 조인 직후 부분 집계(Id 6) → 지역코드 해시로 재분배(Id 5 PX SEND HASH, TQ10002) → 최종 집계(Id 3)의 2단계로 풀립니다. 이 집계용 재분배는 조인용 재분배(TQ10000·10001)와 다른 TQ(TQ10002)에서 일어나는 독립된 단계입니다.

- ② hash-hash 조인 + 별도 집계 재분배 : 조인 SEND(9·13)와 집계 SEND(5)가 다른 TQ → 플랜과 일치  
 ① broadcast : Id 9(:TQ10000)가 PX SEND BROADCAST여야 함 → 실제로는 PX SEND HASH  
 ③ full PWJ : 조인 입력에 SEND가 없어야 함 → Id 9·13에 PX SEND HASH 있음(P→P)  
 ④ 집계 1단계 : Id 5(:TQ10002)는 P→P 재분배지 QC 통로 아님(QC는 Id 2 P→S)

조인 입력 위에 **PX SEND HASH**가 있다는 사실이 broadcast(①)·full PWJ(③)를 배제하고, Id 5·6·3의 2단계 집계 구조가 "집계 재분배 없음/1단계"(④)를 배제합니다. 따라서 ②가 옳습니다.

| 선택지 | 판정 | 이유                                                                                                                                                                                                                                    |
|-----|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | :TQ10000이 붙은 Id 9는 <b>PX SEND BROADCAST</b> 가 아니라 <b>PX SEND HASH</b> 이므로 급수전은 복제되지 않고, 급수사용량도 Id 13에서 <b>PX SEND HASH</b> 로 재분배되어 "재분배 없이 읽는다"가 틀립니다. 집계 역시 Id 5에서 재분배되므로 "서버 로컬로 끝난다"도 성립하지 않습니다                                    |
| ②   | ○  | Id 9·13의 <b>PX SEND HASH</b> (TQ10000·10001)로 두 입력을 조인 키 해시 재분배해 hash-hash 조인하고, 집계는 부분 집계(Id 6) → 지역코드 해시 재분배(Id 5, TQ10002) → 최종 집계(Id 3)로 조인과 다른 TQ에서 별개로 일어납니다                                                                    |
| ③   | ×  | 발문이 어느 쪽도 급수전번호로 파티셔닝돼 있지 않다고 못 박았으므로 대응 파티션 조인 자체가 성립하지 않습니다. Id 9·13은 <b>PX SEND HASH</b> 에 IN-OUT이 <b>P-&gt;P</b> 라 파티션 배정 통로가 아니라 행을 실제로 실어 나르는 재분배이고, full 파티션 와이즈였다면 이 SEND 자체가 없어야 합니다                                         |
| ④   | ×  | 앞부분의 hash-hash 재분배는 맞지만, 집계 키(지역코드)가 조인 키(급수전번호)와 달라 조인 결과는 지역코드로 뭉쳐 있지 않습니다. Id 5는 IN-OUT <b>P-&gt;P</b> 의 재분배이지 QC 통로가 아니며(QC 전송은 Id 2의 <b>P-&gt;S</b> ), Id 6(부분)과 Id 3(최종)은 그 SEND를 사이에 낀 서로 다른 2단계 집계입니다 — 집계 재분배 유무를 뒤바꾼 값스왑입니다 |

#### ▪ 이 문제의 핵심

1. 파티셔닝이 없으면 조인은 hash-hash 재분배입니다. 두 입력을 조인 키(급수전번호) 해시로 각각 재분배(Id 9·13 PX SEND HASH)해 같은 버킷끼리 조인합니다.
2. 집계 키가 조인 키와 다르면 집계 재분배가 따로 필요합니다. 지역코드 집계는 부분 집계(Id 6) → 지역코드 재분배(Id 5) → 최종 집계(Id 3)의 독립 단계로 풀립니다.
3. 조인 재분배와 집계 재분배는 서로 다른 TQ입니다. 조인은 TQ10000·10001, 집계는 TQ10002에서 일어나며 둘은 별개입니다.
4. broadcast·full PWJ는 조인 입력의 SEND 종류로 배제합니다. broadcast면 **PX SEND BROADCAST**, full PWJ면 SEND 자체가 없어야 하는데, 이 플랜의 조인 입력에는 **PX SEND HASH**가 붙어 있습니다 — 분배 방식을 뒤바꾼 값스왑 오답입니다.

- 한 줄 정리 급수전·급수사용량이 조인 키로 파티셔닝돼 있지 않아 Id 9·13 PX SEND HASH로 hash-hash 조인하고, 집계 키(지역코드)가 조인 키와 달라 부분 집계 → Id 5 재분배 → 최종 집계로 조인과 별개의 TQ에서 집계 재분배가 일어난다.

#### 문제 18. 정답 ④

**PGA\_AGGREGATE\_TARGET**은 인스턴스 전체가 나눠 쓰는 총량 목표치이지, 정렬 하나가 통째로 차지할 수 있는 값이 아닙니다.

PGA\_AGGREGATE\_TARGET = 인스턴스 전체 PGA 총량 목표(모든 세션·work area가 나눠 씀)

↳ 단일 work area 한도 = 그 총량의 '일부'로 자동 제한(한 정렬이 전부를 못 가져감)

④ : 정렬 하나가 목표치 '전부'를 work area로 통째로 할당 ← 부분 한도를 전체로 확대

④는 "총량 목표치"라는 전체 값을, 그 안의 한 부분집합인 단일 정렬의 work area 한도에 그대로 끼워 넣었습니다. 실제로 자동 PGA 관리는 동시 부하를 고려해 개별 work area가 쓸 수 있는 크기를 총 목표치의 일부로 제한하므로, 정렬 하나가 전체 목표치를 통째로 할당받지는 못합니다. 부분(단일 work area 한도) → 전체(인스턴스 총량) 확대라 ④가 부적절합니다.

나머지는 모두 옳습니다.

①: 정렬 대상이 work area에 다 담기면 메모리 내 정렬(optimal)로 끝나 추가 디스크 I/O가 없습니다.

②: work area를 넘치면 Temp로 내렸다 병합하는 디스크 소트가 되고, one-pass→multi-pass로 패스가 늘수록 I/O 부담이 커집니다.

③: **PGA\_AGGREGATE\_TARGET**을 정해 두면 서버가 각 세션 work area를 자동 조절하므로 **SORT\_AREA\_SIZE**를 수동으로 잡지 않아도 됩니다.

| 선택지 | 판정 | 이유                                                                                                           |
|-----|----|--------------------------------------------------------------------------------------------------------------|
| ①   | ○  | 정렬 대상이 work area에 다 담기면 메모리 내 정렬(optimal)로 끝나 추가 디스크 I/O가 없습니다                                               |
| ②   | ○  | work area를 넘치면 Temp로 내렸다 병합하는 디스크 소트가 되고, one-pass→multi-pass로 패스가 늘수록 I/O 부담이 커집니다                          |
| ③   | ○  | <b>PGA_AGGREGATE_TARGET</b> 을 정해 두면 서버가 각 세션 work area를 자동 조절하므로 <b>SORT_AREA_SIZE</b> 를 수동으로 잡지 않아도 됩니다     |
| ④   | ×  | 목표치는 인스턴스 전체가 나눠 쓰는 총량이며, 단일 work area 한도는 그 일부로 자동 제한됩니다. 정렬 하나가 목표치 전부를 통째로 할당받겠다는 것은 부분 한도를 전체로 확대한 오류입니다 |

▪ 이 문제의 핵심

1. 정렬은 work area(Sort Area)에서 수행된다. 대상이 다 담기면 메모리 내 정렬(optimal)로 끝나 추가 I/O가 없습니다.
2. 넘치면 디스크 소트, 패스가 관건. one-pass→multi-pass로 병합 패스가 늘수록 Temp I/O 부담이 커집니다.
3. **PGA\_AGGREGATE\_TARGET**은 자동 관리의 총량 목표. 서버가 각 세션 work area를 자동 조절해 수동 **SORT\_AREA\_SIZE** 설정을 대체합니다.
4. 총량 목표 ≠ 단일 정렬의 한도. 개별 work area는 총 목표치의 일부로 제한되며, 정렬 하나가 전체를 통째로 쓰지 못합니다.

▪ 한 줄 정리 **PGA\_AGGREGATE\_TARGET**은 인스턴스 전체가 나눠 쓰는 총량 목표치이고 단일 work area 한도는 그 일부로 제한되므로, "정렬 하나가 목표치 전부를 통째로 할당받는다"는 진술은 부분 한도를 전체 총량으로 확대한 오류입니다.

문제 19. 정답 ③

오라클은 MVCC로 문장 수준 읽기 일관성(statement-level read consistency)을 보장합니다. 하나의 SELECT는 문장이 시작된 시점(t1)의 SCN을 기준으로, 그 시점에 커밋돼 있던 값만 봅니다. 스캔 도중 다른 트랜잭션이 커밋해 블록이 바뀌어도, 블록의 SCN이 기준 SCN보다 크면 오라클은 Undo로 t1 시점 버전을 재구성(CR, Consistent Read 블록)해 읽습니다.

[ TX2 SUM의 읽기 일관성 계산 - 기준 SCN = t1(문장 시작) ]  
스캔 순서 A → B → C

A: t1 이후 변경 없음 → 현재 블록 그대로 = 100000  
B: t2에 250000로 커밋(블록 SCN>t1) → Undo로 t1 버전 재구성(CR) = 200000  
C: t3에 380000로 커밋(블록 SCN>t1) → Undo로 t1 버전 재구성(CR) = 300000

SUM = 100000 + 200000 + 300000 = 600000

(만약 해당 Undo가 덮어써졌다면 CR 재구성 실패 → ORA-01555. 단서로 배제)

핵심은 기준 시점이 문장 시작(t1)에 고정된다는 것입니다. B·C가 t2·t3에 커밋됐어도 그 커밋 SCN은 t1보다 뒤이므로, TX2는 각 블록을 Undo로 t1 시점(200,000·300,000)으로 되감아 읽습니다. 따라서 SUM = 600,000, 정답은 ③입니다.

①②④는 커밋된 최신값 일부 또는 전부를 섞어 만든 값스왑 오답입니다. ①은 두 커밋을 모두 본

730,000(=100000+250000+380000), ②는 B만 반영한 650,000, ④는 C만 반영한 680,000 — 모두 문장 시작 시점 고정을 어긴 조합입니다.

| 선택지 | 판정 | 이유                                                                      |
|-----|----|-------------------------------------------------------------------------|
| ①   | ×  | 100000+250000+380000. 두 커밋을 모두 반영한 현재값 읽기로, 문장 시작(t1) 스냅샷을 어깁니다         |
| ②   | ×  | 100000+250000+300000. B만 갱신값(250000), C는 원래값으로 섞은 값스왑입니다                |
| ③   | ○  | 기준 SCN=t1. B·C 블록은 Undo로 t1 버전(200000·300000)을 CR 재구성해 읽어 SUM=600000입니다 |
| ④   | ×  | 100000+200000+380000. C만 갱신값(380000), B는 원래값으로 섞은 값스왑입니다                |

▪ 이 문제의 핵심

1. 오라클 SELECT는 문장 시작 시점의 SCN을 기준으로 읽습니다(문장 수준 읽기 일관성). 스캔 도중 커밋된 값은 반영하지 않습니다.
2. 블록 SCN이 기준보다 크면 Undo로 되감습니다. 과거 버전을 재구성한 것이 CR(Consistent Read) 블록이며, 그 결과 t1 시점 값(200000·300000)을 읽습니다.
3. SUM = 600,000. A는 그대로, B·C는 CR 재구성으로 초기값을 더해 일관된 스냅샷을 반환합니다.
4. Undo가 덮어써지면 ORA-01555(snapshot too old)가 납니다. CR 재구성에 필요한 Undo가 남아 있어야 일관된 읽기가 성립합니다.

▪ 한 줄 정리 오라클은 SELECT 시작 시점(t1) SCN을 기준으로 읽으므로, 스캔 중 커밋된 B·C도 Undo로 t1 버전을 CR 재구성해 SUM=600,000이 되고, 정답은 ③입니다.

문제 20. 정답 ④

잠금 에스컬레이션(lock escalation)은 잠금을 잘게(fine) → 굵게(coarse)로 올리는 최적화입니다. 한 문장이 확보한 개별 잠금이 너무 많아지면(대략 5,000개 기준) SQL Server가 그 다수의 행·페이지 잠금을 하나의 테이블(OBJECT) 잠금으로 대체해 잠금 관리 부하를 줄입니다.

방향: 행(KEY/RID) · 페이지(PAGE) — (개수 급증 ~5000) —▶ 테이블(OBJECT)

= 잘게 → 굵게 (coarsening), 잠금 개수↓ · 관리비용↓ · 동시성↓

표 해석:

SID 58 : OBJECT X GRANT → 테이블 전체를 배타로 보유(막는 중)  
 SID 72 : OBJECT IS WAIT → X와 비호환 → 대기(블로킹)  
 SID 80 : OBJECT S WAIT → X와 비호환 → 대기(블로킹)

④는 SQL Server의 잠금 동작을 정반대로 뒤집었습니다. 에스컬레이션은 "테이블→페이지→행으로 잘게 쪼개 내려가는" 것이 아니라 행·페이지 → 테이블로 굵게 올리는 방향입니다. 게다가 테이블에 X 잠금이 걸리면 다른 세션의 요청(IS·S)은 X와 호환되지 않아 블로킹됩니다 — 표의 72·80이 WAIT인 이유가 바로 그것입니다. ④는 방향(굵게↔잘게)과 효과(블로킹 발생↔안 함) 두 가지를 모두 거꾸로 말했으므로, 부적절한 진술은 ④입니다.

①②③은 각각 X 테이블 잠금이 IS·S 요청을 블로킹한다는 점(①), 에스컬레이션이 잠금 개수 급증(~5,000)에 대응해 테이블 잠금으로 대체한다는 점(②), 그 대가로 잠금 개수는 줄지만 동시성이 낮아진다는 점(③)을 옳게 서술합니다.

| 선택지 | 판정 | 이유                                                                             |
|-----|----|--------------------------------------------------------------------------------|
| ①   | ○  | 58이 OBJECT X를 보유하므로 IS·S를 요청한 72·80은 X와 비호환이라 WAIT로 블로킹됩니다. 표의 세 행 그대로입니다      |
| ②   | ○  | 에스컬레이션은 한 문장의 잠금이 ~5,000개를 넘으면 이를 하나의 테이블 잠금으로 대체해 관리 부하를 줄이는 최적화입니다           |
| ③   | ○  | 테이블 잠금으로 대체되면 개별 행·페이지 잠금이 해제되어 개수는 줄지만, 굵은 잠금 하나가 테이블을 막아 동시성은 낮아집니다          |
| ④   | ×  | 에스컬레이션은 행·페이지→테이블로 굵게 올리는 방향이며(테이블→행이 아님), 그 결과 다른 세션을 블로킹합니다. 방향·효과가 모두 반대입니다 |

#### ■ 이 문제의 핵심

1. 잠금 에스컬레이션은 행·페이지 → 테이블로 굵어지는 방향입니다. 한 문장의 잠금 개수가 급증(~5,000)할 때 관리 부하를 줄이려 발동합니다.
2. 테이블 X 잠금은 IS·S 요청과 비호환이라 다른 세션을 블로킹합니다. **sys.dm\_tran\_locks**에서 한 세션 OBJECT X GRANT, 다른 세션 WAIT가 그 징표입니다.
3. 에스컬레이션의 대가는 동시성 저하입니다. 잠금 개수는 줄지만 굵은 잠금 하나가 테이블 전체를 막습니다.
4. 에스컬레이션은 개수에 반응하는 메커니즘일 뿐, 잠금을 잘게 쪼개 내려가거나 블로킹을 없애는 기능이 아닙니다.

■ 한 줄 정리 SQL Server 잠금 에스컬레이션은 행·페이지→테이블로 굵어지며 그 테이블 X 잠금이 다른 세션을 블로킹하므로, "테이블→행으로 잘게 내려가 블로킹이 없다"는 ④는 방향·효과가 모두 반대입니다.